



X01: From x_0 to x_1

On Learning Flow Dynamics in Two-Dimensional Kolmogorov Flow using
Koopman Operator Theory and Diffusion-Based Vorticity Reconstruction

Efe Şahin¹

M.Sc. in Machine Learning

Dr. Xiao Xue, Dr. Ira Shokar, Professor Peter V. Coveney

Submission date: 8 September 2026

¹**Disclaimer:** This report is submitted as part requirement for the M.Sc. in Machine Learning at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged

Abstract

This thesis studies a surrogate model for two-dimensional Kolmogorov-flow vorticity prediction that separates the task into an autoregressive module for low-resolution temporal rollout and a super-resolution module for high-resolution spatial reconstruction. The central question becomes whether the temporal component can be represented by a Koopman-style autoencoder, where a finite latent operator advances the encoded state forward in time.

A new Kolmogorov-flow dataset was generated to focus the experiments on post-transient, chaotic regimes. The autoregressive module was evaluated through rollout visualizations, mean squared error, temporal autocorrelation, energy spectra, Q–Q diagnostics, and Koopman-operator spectral statistics. The results show that strict single-operator Koopman rollouts are sensitive to model capacity, loss weighting, checkpoint selection, and inference mode. Orthogonality regularization improved stability, but did not solve the full long-horizon rollout problem.

The super-resolution module was implemented as a conditional diffusion model and was able to reconstruct coherent high-resolution vorticity structures from clean low-resolution conditions. When the two modules were coupled, the resulting AR–SR pipeline produced high-resolution rollouts end-to-end, but its accuracy remained limited by the low-resolution temporal predictions. The thesis therefore presents the modular AR–SR pipeline as useful and executable, while identifying the strict finite Koopman operator as the main bottleneck. The code-base is available at github.com/Efesasa0/x01.

Acknowledgements

This work would not be possible without the supervision of Professor Peter V. Coveney, Dr. Xiao Xue and Dr. Ira Shokar, which formed my entryway back into physical sciences after a long period of looking for one.

I also acknowledge the computational resources used throughout this project, including GPU access through UCL resources, Isambard 2, and personally funded cloud GPU time through Run-Pod. These resources made the experimentation in this thesis possible.

Most importantly, my deepest thanks to my family and my cat, Kediko, for his availability to have meetings with me for emotional support, while being busy wandering the streets of Istanbul.

Contents

1	Introduction	9
2	Related Work	12
3	Background	17
3.1	Kolmogorov Flow and Vorticity Fields	17
3.2	Autoregressive Module	21
3.3	Super-Resolution Module	30
4	Method	36
4.1	Problem Setup	36
4.2	Dataset	37
4.2.1	Initial Dataset	37
4.2.2	Generated Dataset	37
4.2.3	Preprocessing and Splits	39
4.3	AR Module	40
4.3.1	Training Objective	40
4.3.2	Inference Modes	45
4.3.3	Evaluation	45
4.3.4	Latent Least-Squares Probe	48
4.4	SR Module	48
4.4.1	Training Objective	49
4.4.2	Evaluation	51
4.5	Joint Module	52
4.6	Implementation	53
5	Experiments and Results	54
5.1	Dataset Experiments	54
5.2	Autoregressive Module Experiments	57
5.2.1	Exploratory Experiments	57
5.2.2	Latent Least Squares Probe	61
5.2.3	Ablations under Strict Single-Operator Rollout	64
5.2.4	Sequential inference	65
5.2.5	Additional AR Experiments	66

5.3	Super-Resolution Module Experiments	70
5.4	Joint Module Experiments	72
6	Discussion and Limitations	75
7	Conclusion	77
A	Use of AI Tools	78

List of Figures

3.1	A glass of Turkish tea as a simple depiction of vorticity assuming three dimensions of space. The local swirling curls represent vorticities localized on infinitesimal chunks of space within the glass of tea.	18
3.2	Conceptual sketch of a vorticity field, along the time axis. Each field represents the same domain and the front field shows the field $\omega_t(x, y)$ at time t assuming continuous-time physics. The signed vortical patches indicate a local rotation that can take both positive and negative values.	19
3.3	Conceptual decomposition of the four terms influencing vorticity field evolution in continuous-time t . The four terms are: advection, diffusion, drag and forcing. The arrows point to the center depicting how they could interact visually to produce the next vorticity state.	20
3.4	Conceptual depiction of a vorticity field at time t along continuous-time dynamics. The domain $D \subset \mathbb{R}^2$ contains spatial coordinates, while the full state $x(t) = \omega(\cdot, \cdot, t)$ assigns one scalar value to each point, with $x(t)(x_i, y_j) \in \mathbb{R}$. The edge lines indicate the whole-domain to whole-field relation, while the marked point shows one pointwise evaluation.	22
3.5	Conceptual depiction of Cauchy sequences in a complete Hilbert space. The sequence elements, which may be functions such as vorticity fields, become arbitrarily close to one another, and the limiting element remains inside $\mathcal{H}$	24
3.6	Conceptual sketch of Koopman linearity. The left panel shows nonlinear evolution of vorticity states depicted by curved arrows. The right panel keeps the same curved state trajectory underneath, while the observable evaluations above it are advanced linearly by the Koopman operator $\mathcal{K}_{\Delta t}$	27
3.7	Conceptual sketch of a Koopman-invariant observable subspace. The full Koopman operator acts on the observable space, but observables inside the finite span of eigenfunctions remain inside the same subspace under repeated evolution. Each eigenfunction direction changes only by its corresponding spectral multiplier. . . .	28
3.8	Conceptual sketch of the SR module. The low-resolution frame is upsampled to form the condition c_k . Starting from a noisy high-resolution sample $x_{k,\tau}$, the diffusion sampler iteratively removes predicted noise through intermediate diffusion states such as $x_{k,\tau'}$, while using the condition, producing the high-resolution reconstruction $\hat{x}_k^{HR}$	31

3.9	Conceptual sketch of the diffusion noising and denoising chain used for the SR module. The upper row shows the training construction, where the clean high-resolution target $x_{k,0} = x_k^{HR}$ is gradually noised into intermediate noisy states $x_{k,\tau}$ and then toward the final noisy state $x_{k,\tau_{\max}}$. At sampling time, the reverse chain is initialized from a near-Gaussian state $x_{k,\tau_{\max}}$ and denoised through selected DDIM steps while being conditioned on the upsampled low-resolution frame c_k	33
5.1	Example trajectories from the initial and generated datasets. The top row shows snapshots from one trajectory in the initial dataset, while the bottom row shows snapshots from one trajectory in the newly generated dataset.	55
5.2	Dataset filtering and comparison diagnostics. (a) Comparison of 10 representative trajectories from each dataset used to compare temporal spread. (b) Histogram of eliminated trajectories from the newly generated dataset. (c) Comparison of the mean autocorrelation plots of both datasets. (d) Comparisons of the mean energy spectra plots of both datasets.	56
5.3	Validation rollouts for the numerical instability case. The rows compare the ground truth rollout with predictions from epochs 20, 40 and 80 at selected time steps. . .	58
5.4	Diagnostics for the numerical instability case. (a) the evolution of the forward operator spectrum. (b) the training and validation losses. (c) the rollout vorticity magnitude across saved epochs.	59
5.5	Diagnostics for the artifact-propagation case. (a) the forward operator spectrum of the learned forward K operator. (b) the training and validation losses from learning. (c) the magnitude of maximum rollout vorticities generated by AR across saved epochs.	59
5.6	Validation rollouts for the artifact-propagation case. The rows compare the ground truth rollout with predictions from epochs 20, 40 and 80 at selected time steps. Unlike the numerical-instability case, the predictions remain bounded, but later frames develop persistent spatial artifacts.	60
5.7	Orthogonality constrained AR rollout diagnostics. (a) Ground truth trajectory and epoch-80 prediction at selected time steps. (b) Mean validation rollout MSE across saved epochs. (c) Minimum, mean, and maximum eigenvalue magnitudes of the learned forward operator K . (d) Training and validation loss curves.	61
5.8	Least-squares latent linearizability probe. A single finite-dimensional forward operator K^* is fitted by least squares on frozen reconstruction-only autoencoder latents using increased dataset size with $N = (1, 2, 8, 40)$ training trajectories.	62
5.9	Qualitative PCA projections of latent states from independently trained reconstruction-only autoencoders. Points are colored by normalized time step.	63
5.10	Latent covariance spectra for reconstruction-only autoencoders. The left panel shows the normalized covariance eigenvalue spectrum of the encoded latents sorted by rank where the right panel shows the cumulative explained variance.	63

5.11	Diagnostics comparison of the three ablations. Rows show the strict single- K baseline, the latent-step consistency variant and the increased-capacity variant. Columns compare temporal autocorrelation, energy-spectrum, Q-Q statistics, forward operator spectra, and training curves.	65
5.12	Sequential AR inference on the generated test set. The top rows compare ground-truth and AR-predicted low-resolution vorticity snapshots at selected time steps. The bottom row shows temporal autocorrelation, energy-spectrum and Q-Q diagnostics.	66
5.13	Stage 1 capacity sweep for the strict- K AR model. Rows show encoder-decoder profiles, columns show base width, and cells report the validation autocorrelation score S_R above the total parameter count.	67
5.14	Stage 2 continuation sweep for the selected Stage 1 candidates. Rows show candidate capacity settings, columns show epochs, and cells show validation S_R	68
5.15	Stage 3 loss-coefficient sweep for the selected alpha_e2_d064 capacity setting. Rows show coefficient configurations sorted by best validation S_R , and columns show epochs.	69
5.16	Ranking of the Stage 3 loss-coefficient sweep. The left panel compares the best validation S_R with the final-epoch score for each configuration, while the right panel lists the corresponding forward, backward, reconstruction, latent-forward and orthogonality coefficients.	70
5.17	SR reconstruction samples. Rows correspond to selected test frames. Columns show the high-resolution reference, the upsampled low-resolution condition, the DDIM super-resolution sample, and the absolute error.	71
5.18	Energy-spectrum diagnostic for the SR module on selected test-set frames. The curves compare the high-resolution reference, the upsampled low-resolution condition, and the DDIM super-resolution sample.	72
5.19	Diagnostics for the joint AR-SR pipeline. The image rows show the low-resolution reference rollout, the low-resolution AR rollout, and the high-resolution AR-SR rollout. The lower panels report high-resolution autocorrelation, energy-spectrum $E(\kappa)$, and Q-Q diagnostics over the full rollout.	73
5.20	Long-horizon rollout from the joint AR-SR pipeline. The panels show selected high-resolution vorticity fields generated by applying the SR module to the AR rollout at increasing time steps.	74

List of Tables

4.1	Main construction details for the generated Kolmogorov-flow dataset.	38
4.2	Trajectory-level train, validation and test splits used for the AR and SR modules. Both splits follow an approximately 80/10/10 split of the 314 valid generated tra- jectories.	39
4.3	AR architecture configurations used in the experiments.	44
4.4	SR architecture configuration used in the experiments.	50
5.1	Trajectory-level splits and corresponding training, validation, testing sample counts used after filtering the generated dataset.	57

List of Algorithms

1	Kolmogorov flow dataset generation	38
2	Koopman autoencoder training	44
3	Conditional diffusion super-resolution training	50
4	Joint AR-SR rollout	52

Chapter 1

Introduction

In this thesis, the central idea can be stated in simple terms, that is, there exists an initial state, denoted by x_0 , and there exists a subsequent state after one deterministic time step, denoted by x_1 . The name X01 reflects this basic transition from x_0 to x_1 .

The question is therefore what rule, if any, governs prediction between these two objects, that is, what carries the first state into the next state. For simple systems, this rule may be written down and solved directly. For turbulent flows, with which this thesis is concerned, the rule exists in the governing equations, but its visible behaviour is distributed across many interacting scales of complexity. A small local change in the vorticity field may later appear as a global change in the rollout. Thus, even the apparently simple transition from x_0 to x_1 becomes a question about representation, stability, and approximation. This thesis therefore asks how far such a transition can be learned when the temporal model is deliberately constrained to act through a finite Koopman-style latent operator.

Longer descriptive names for this project could have emphasized specific technical components, such as KARRSS for Koopman Aided Regression of Reconstructed Systems, or KODR for Koopman Operator aided Diffusion based Reconstructions. However, such names would obscure the central point of the notation.

Within this framing, this thesis concerns two-dimensional Kolmogorov-flow vorticity fields, where each field is treated both as a physical object and as a computational object. From the physical perspective, the vorticity is a continuous field, that is, a function over space. From the computational perspective, the same object is represented only through a finite array of floating point values in finite memory.

This mismatch is not only a technical inconvenience, since it defines the space in which the problem lives. The physical system evolves as a continuous object, but the learned model can only act on a finite representation of it. Therefore, even before learning begins, the question from x_0 to x_1 is already a question about what representation is allowed to carry the dynamics.

The physical and computational perspectives therefore do not fully coincide. If the full fidelity of a vorticity field cannot be represented in a computer, then the transition dynamics between fields must also be studied through a finite representation. In this thesis, the central problem is also concerned with this gap, where the physical description seeks precision, while the computational model can only operate on a finite representation when a derivation or theory needs to be field

tested.

The computational perspective usually uses tools called numerical solvers to describe dynamical transitions. This thesis is not primarily concerned with designing numerical solvers that provide accurate rollouts by solving a particular set of governing equations, namely ordinary differential equations and partial differential equations. Rather, it is concerned with how methods in this space of physics for computation have evolved toward learned models. High-fidelity computational fluid dynamics (CFD) also remains computationally demanding, which motivates surrogate models that can reduce repeated simulation cost while retaining useful physical behaviour [Slotnick et al., 2014].

Recent approaches in this space often use neural models, which are inspired by neuroscience but must also be represented within the computational setting. A useful property of neural models is that, through supervised learning, they can learn an internal system of coordinates for solving a hand-designed objective. When these models are made deep, that is, when more layers, neurons, and connections are introduced, this internal coordinate system can become expressive enough to represent complicated transitions [Goodfellow et al., 2016].

In this thesis, the problem of learning the transition between the simple states x_0 and x_1 is separated into two connected problems. One neural model learns the transition dynamics while sacrificing spatial precision in the represented vorticity field, whereas the other sacrifices the temporal view in order to enhance spatial precision and recover the level of detail required for physical interpretation.

The supervised learning procedure requires a collection of examples to learn from, where the neural model optimizes toward a set of objectives designed to complement one another. Such data is often acquired from previous methods, namely by using numerical solvers themselves. For the two neural models mentioned above, the first is called the autoregressive (AR) module, whereas the second is called the super-resolution (SR) module [Xue et al., 2026]. The AR module specifically utilizes a physics-inspired framework, namely Koopman linearizability theory [Koopman, 1931, Koopman and von Neumann, 1932]. The SR module utilizes a probabilistic approach, namely diffusion probabilistic modelling [Ho et al., 2020, Song et al., 2021, Saharia et al., 2023], during its learning procedure.

The AR module is tasked with learning an encoder, a type of neural model architecture used in the field, which produces meaningful coordinate spaces from provided inputs, such as vorticity images or frames in low-resolution space. By using low resolution, the problem of precision is addressed through a reduced computational space, where the representation is less detailed but more tractable for learning the transition dynamics. In contrast, the SR module is tasked with learning a map from the low-resolution space of images to a higher-resolution space of images.

The central thesis, however, targets the AR module itself, as the whole system is bounded by it. Meaningful dynamics are required to generate meaningful trajectories, whether in low-resolution or high-resolution space, and therefore the SR module serves as an additional module of choice.

The AR module is simple in idea, although it leverages Koopman operator theory. The Koopman viewpoint shows that nonlinear state dynamics, such as Kolmogorov-flow dynamics, can be studied through a linear operator acting on a different type of function called an observable, although the exact operator generally lives in an infinite-dimensional function space. This idea is attractive because it translates naturally to fluid dynamics, where transition dynamics can be explained through the observable function space on which the operator acts linearly. From a

neural model perspective, the idea also becomes direct, that is, the neural model learns a finite-dimensional coordinate space of observables on which a square matrix acts linearly to obtain the next coordinate space of observables, from where the neural model transforms it back to the image space of vorticity frames. The learned coordinate space is often called the latent representation space, which is finite-dimensional, while the learned Koopman operator is represented by a finite matrix. The question regarding the AR module therefore becomes whether a finite learned operator can carry enough of the turbulent dynamics to produce stable rollouts when the AR module is applied iteratively, predicting the next latent states through repeated linear transformation by the learned Koopman operator. Thus, this is where ideas such as dynamical systems, fluid mechanics and representation learning meet. Related methods such as dynamic mode decomposition, Koopman autoencoders, and neural operators form the surrounding landscape for this question [Schmid, 2010, Rowley et al., 2009, Lusch et al., 2018, Otto and Rowley, 2019, Azencot et al., 2020, Li et al., 2024].

The SR module is also simple in idea, although it is supported by theoretically involved interpretations. It leverages a probabilistic perspective because, for a given low-resolution field, there may be many explainable higher-resolution counterparts. The objective therefore becomes finding a preferable counterpart under the supervised data setting it has been provided. The diffusion process is attractive because it does not follow an instantaneous process, but rather a step-wise one, where starting from pure Gaussian noise, the model learns to iteratively nudge the sample toward a statistically reasonable part of the image space that resembles the original data. The chosen configuration uses a probabilistic approach during training, whereas a deterministic technique is used during inference. In this context, inference refers to using the model to generate data, namely high-resolution reconstructions of low-resolution vorticity frames. The model is therefore not intended to generate new data blindly, but to use information from the AR module, which is summarized as the need to learn a conditional model. Following denoising diffusion probabilistic models, denoising diffusion implicit models and the SR3 line of work [Ho et al., 2020, Song et al., 2021, Saharia et al., 2023], the SR module learns to denoise high-resolution frames while being grounded on the upsampled outputs of the AR module.

The final aim is to connect these two views into a modular AR–SR pipeline referred to as the joint module. In this thesis, the experimental work constructs a larger dataset, studies several Koopman-style AR training regimes, diagnoses failure modes and stabilizing choices, evaluates a conditional SR module, and finally tests the AR–SR pipeline beyond the data horizon. The results show that finite Koopman-style operators are highly sensitive to capacity, loss balance, and inference mode. The joint pipeline is therefore feasible, but its long-horizon accuracy remains limited by the temporal AR stage.

The rest of this thesis is organized as follows. Chapter 2 places the work in relation to Koopman theory, neural dynamical systems, neural operators, and diffusion-based flow reconstruction. Chapter 3 introduces the physical, dynamical-systems, and probabilistic background needed for the AR and SR modules. Chapter 4 defines the dataset, models, training objectives, inference modes, and evaluation diagnostics. Chapter 5 reports the experimental results for the dataset, AR module, SR module, and joint pipeline. Finally, Chapter 6 discusses the main limitations of the learned AR–SR pipeline.

Chapter 2

Related Work

The main idea introduced by Bernard O. Koopman [Koopman, 1931] was the mathematical origin point, as it showed that a classical Hamiltonian flow can be represented as a linear unitary transformation by an infinite-dimensional operator acting on functions of the state. This operator is now called the Koopman operator, and these functions are called observables, which live in a Hilbert space. The induced Koopman operator was shown to be unitary, meaning that it can be studied through the spectral theory of Hilbert space operators. This is the origin of the modern appeal of Koopman methods, where nonlinear dynamics may become linear after lifting the state to a suitable function space. The computational difficulty, which appears later in the literature, is that this function space is generally infinite-dimensional.

The idea was further developed a year later with von Neumann [Koopman and von Neumann, 1932], where the spectral structure of the induced operator was studied more directly. In this setting, spectral analysis means asking what kinds of temporal components the operator contains. Some components may appear as isolated modes, corresponding to clean oscillations or repeating structures, whereas other components may be spread over a continuous spectrum. For systems with continuous spectrum, the exact influence of the initial state can become less informative over long horizons, and the behaviour is therefore often described through statistical structure rather than through exact trajectory prediction alone.

After the original work by Koopman and von Neumann, later work by Mezić and Banaszuk brought the Koopman view closer to physical modelling [Mezić and Banaszuk, 2004]. Here, a model can be understood as a simplified description of the real system, namely, it may be a set of equations, a reduced representation, or a learned rule that describes how the state changes over time. The important shift was that such a model does not always need to reproduce the exact same trajectory at every time in order to be useful. For complex systems, it can also be useful to ask whether the model captures the same long-time patterns seen in the data. Mezić then connected this idea to model reduction, where the goal is to describe a high-dimensional system through a smaller set of components [Mezić, 2005]. This is the route by which the Koopman framework starts to move from Hilbert-space theory toward practical data-driven methods such as DMD.

In fluid mechanics, this operator viewpoint became closely connected to Dynamic Mode Decomposition (DMD), which is a snapshot-based method that separates time-ordered data, such

as fluid flow fields, into spatial patterns and their associated temporal behaviour by using linear algebra. This is useful because each extracted pattern can be linked to an oscillation, growth, or decay seen in the flow. Schmid presented DMD as a practical data-driven procedure for numerical and experimental flow fields, where a low-dimensional linear map is estimated directly from successive snapshots [Schmid, 2010]. Rowley, Mezić and co-authors then showed that modes extracted from nonlinear flow snapshots can be interpreted through the Koopman operator [Rowley et al., 2009]. In this sense, the decomposition is organized by temporal behaviour rather than only by energy, as in Proper Orthogonal Decomposition (POD). This made the Koopman view more familiar in fluid mechanics, since the resulting modes could be compared with global stability modes, Fourier modes, and POD modes. In this way, DMD became one of the main computational bridges between measured flow data and Koopman-style spectral analysis.

This connection was later placed into a broader applied Koopman framework by Budišić, Mohr and Mezić [Budišić et al., 2012]. Their work acted as a survey study, making explicit that Koopman analysis is not only a theory of abstract operators, but also a way of studying measured or simulated data through observables. In particular, they describe how Koopman mode analysis can be made computational by following a finite set of observables along trajectories and then approximating the action of the operator on that finite space. This is also where the practical limitation appears, namely, the computed modes are not the full Koopman operator, but depend on the observables, the data, and the finite projection used. The next question is therefore how this finite observable space should be chosen.

The next step was to ask how this finite-dimensional approximation should be decided. Extended DMD (EDMD) was introduced by Williams, Kevrekidis and Rowley as a way to approximate the Koopman operator on a dictionary of observables [Williams et al., 2015]. Compared to DMD, the main change is that the linear operator is no longer fitted only on the raw state, but on a lifted set of functions of the state. This makes the method more flexible, while also making the choice of dictionary a central modelling decision. Li et al. pushed this point further by learning the dictionary from data with a neural model, instead of fixing it completely in advance [Li et al., 2017]. They emphasized that if the dictionary was poorly constructed, the resulting linear operator may not capture the dynamics well, especially for high-dimensional or highly nonlinear systems. Klus et al. placed these ideas in a broader transfer-operator and model reduction setting through their survey study [Klus et al., 2018]. Their point was that methods developed under different names, such as DMD from fluid mechanics, TICA and VAC from molecular dynamics [Molgedey and Schuster, 1994, Noé and Nüske, 2013, Nüske et al., 2014], and EDMD, shared a common structure. That structure was that they used data pairs to approximate how an operator evolves observables, making EDMD part of a larger numerical framework rather than an isolated extension of DMD. Their discussion also emphasized that more expressive basis functions that act as observables could make a feasible approximation, but in high-dimensional systems they can also become very expensive, ill-conditioned, or prone to overfitting. Related neural directions in molecular dynamics also used time-lagged learning to make the coordinate choice trainable. Wehmeyer and Noé used time-lagged autoencoders to learn slowly evolving coordinates from trajectory data [Wehmeyer and Noé, 2018], while Mardt et al. introduced VAMPnets, where neural networks are trained through a variational score to learn coordinates useful for molecular kinetics [Mardt et al., 2018]. These works do not address the same fluid-rollout problem considered in this thesis, but

they support the same broader point, that is, the coordinate space used for dynamics is itself part of the modelling problem.

In parallel, Koopman-invariant subspaces and lifted linear predictors were studied as routes toward prediction and control, which is a related but different field. Brunton et al. examined when a finite set of observables can form a Koopman-invariant subspace, so that the nonlinear dynamics can be represented by a finite linear model [Brunton et al., 2016]. This is useful for control, but it also shows a limitation, that is, such exact finite representations are restrictive and depend strongly on the chosen observables. Korda and Mezić then used EDMD-style lifting to build linear predictors for nonlinear controlled systems and applied them to model predictive control [Korda and Mezić, 2018]. Together, these works point to the same practical issue, where the usefulness of the fitted linear operator depends on the coordinates, observables, or lifted space on which it acts.

This difficulty motivated deep Koopman methods utilizing deeper neural model architectures, where the lifted coordinates are learned rather than fixed by hand. Takeishi et al. learned observables for which a linear DMD-style regression fits well, while Yeung et al. used deep neural models to learn the dictionary and Koopman operator together [Takeishi et al., 2017, Yeung et al., 2017]. Morton et al. then applied a related Deep Koopman model to unsteady cylinder flow and control, where an encoder and decoder were learned while the latent dynamics matrix was fitted through least squares and applied recursively during training [Morton et al., 2018]. This is closer to the fluid setting of this thesis, although the problem, training procedure and use of the latent operator are still different. The same idea was then made more explicit through autoencoder-based Koopman models, where Lusch et al. learned the coordinates in which the dynamics were able to evolve linearly, using reconstruction, forward prediction and latent-linearity losses, while Otto and Rowley used a linearly recurrent autoencoder to learn the low-dimensional latent space whose time evolution was made linear [Lusch et al., 2018, Otto and Rowley, 2019]. These works are important because they turn the choice of observables into a trainable part of the model and, at the same time, make the main approximation issue more visible. The issue here is that the learned latent space must remain useful not only under single-step prediction, but also under repeated applications of the learned linear operator.

More recent neural Koopman work has therefore focused on the learned latent operator and the representation methods for acquiring a Koopman-invariant subspace under repeated rollout dynamics. Azencot et al. introduced consistent Koopman autoencoders, where forward and backward latent operators are learned separately and regularized so that they behave as approximate inverses [Azencot et al., 2020]. This is important because it treats long-horizon forecasting as an invertible problem. Kaiser et al. made a similar argument from the eigenfunction perspective, showing that finite Koopman approximations may produce spurious eigenfunctions when the chosen observable space is not closed under the operator [Kaiser et al., 2021]. A second line of recent work adapts Koopman representations to more specific modelling tasks. Li et al. used graph neural models to learn object-centric Koopman embeddings for deformable systems such as ropes and soft robots, where the linear operator is given additional block structure for prediction and control [Li et al., 2020]. Falconer et al. combined Koopman autoencoders with ensemble Kalman filtering so that the latent state and model parameters can be updated as new observables arrive [Falconer et al., 2025]. Boscariol et al. similarly combined a learned Deep-Koopman representation

with a Kalman-filter observer for nonlinear multibody systems, showing another use of Koopman coordinates for state estimation rather than direct rollout forecasting [Boscariol et al., 2025]. Wu et al. moved in a probabilistic direction by combining Koopman and Kalman components using a variational autoencoder (VAE) for long-horizon probabilistic rollout predictions [Wu et al., 2025]. Nayak et al. introduced temporal consistency regularization, where different latent predictions that should arrive at the same future time were encouraged to be similar, while improving robustness with limited or noisy data [Nayak et al., 2025]. For high-dimensional flows, Halder et al. used a Koopman-inspired loss in a beta-variational autoencoder to denoise turbulent latent variables before forecasting them with an LSTM ensemble [Halder et al., 2025], while Grozavescu et al. studied continuous-time Koopman autoencoders for fluid forecasting and showed that rollout training, forward-backward consistency, latent regularization and physical conditioning were needed for stable long-horizon behaviour [Grozavescu et al., 2026]. Taken together, these works show a common trend, that is, once a finite Koopman model is used for long rollouts, noisy observations, control, or high-dimensional physical states, the learned linear operator usually requires extra structure around it.

Alongside deep Koopman models, neural operators provide another route for learning surrogate models of fluid flows. The basic shift is that the network is not only trained as a map between two finite vectors, but as a map between fields, such as from an initial condition or coefficient field to a solution field. The Fourier Neural Operator (FNO) makes this practical by doing part of this mapping in Fourier space, where large-scale spatial interactions can be represented through spectral convolution [Li et al., 2021]. Here, spectral implies spatial frequency content in the Fourier sense rather than a Koopman spectrum of a temporal evolution operator. Physics-informed neural operators (PINO) then add the governing equation back into this operator-learning setting. In PINO, the model is still trained as a neural operator, but its predictions are also penalized when they violate the PDE initial conditions, boundary conditions, or related physical constraints [Li et al., 2024]. This is close to the present thesis in the broad sense that both are concerned with learning fluid dynamics from data. However, the modelling commitment of neural operators is different, namely, they use powerful nonlinear mappings instead of requiring a linearized solution in the latent space.

Diffusion models form the second methodological background of this thesis, namely, the use of Denoising Diffusion Probabilistic Models (DDPM) for reversing a gradual noising process. DDPMs made this idea practical by training a neural model to remove Gaussian noise step by step [Ho et al., 2020], while Denoising Diffusion Implicit Models (DDIM) showed that the same denoising model can be sampled through a faster and more deterministic procedure [Song et al., 2021]. Therefore, such models are suitable for refinement problems, where the goal is not only to predict a coarse state, but to recover plausible missing details. The ideas of refinement and super-resolution were also explored through SDEdit and SR3 [Meng et al., 2022, Saharia et al., 2023]. Similar ideas have also been used for scientific flow fields. Shu et al. used a diffusion model to reconstruct high-fidelity turbulent flow data from lower-fidelity or sparse inputs, and also showed that physics-based information such as PDE residuals can guide the reconstruction [Shu et al., 2023]. This motivates the separation used in the present thesis. Namely, the temporal model is responsible for advancing the coarse state, whereas the SR module is responsible for recovering spatial detail from that state. Therefore, the diffusion part is not introduced as a replacement for Koopman dynamics, but as a

reconstruction mechanism placed after the temporal prediction problem.

Recent autoregressive-diffusion flow models have combined these two roles more directly, as in Uni-Flow, where a low-resolution autoregressive model advances the temporal data forward in time, while a diffusion model is then used to recover the corresponding high-resolution flow fields [Xue et al., 2026]. In the two-dimensional Kolmogorov-flow case, however, their work focuses on using a Fourier Neural Operator trained to predict the next low-resolution vorticity frame, while the Koopman-based latent linear model is used in other Uni-Flow settings, such as turbulent inflow generation and stenotic aortic flow. This distinction matters for the present thesis because the AR–SR separation is similar, but the temporal model studied here is deliberately restricted to a Koopman-style finite latent operator on the Kolmogorov-flow problem itself. In addition, the SR module here is treated as a conditional diffusion generator rather than only an SDEdit-style refinement stage, so that the low-resolution or physics-guided information enters the denoising model more explicitly.

Other recent work also shows that learned rollout and reconstruction pipelines are becoming a broader direction in fluid modelling. Kohl et al. benchmark autoregressive conditional diffusion models for turbulent flow simulation, where diffusion is used as part of the temporal prediction model itself [Kohl et al., 2026]. Hemmasian and Barati Farimani study a coupled encoder-diffusion model, where a flow field is compressed and a conditional diffusion decoder reconstructs statistically and spectrally faithful fields from that compressed representation [Hemmasian and Barati Farimani, 2026]. These works help locate the present thesis locally. The aim here is not to replace the full range of neural operators, diffusion models, or modern Koopman autoencoders. Instead, the thesis studies a narrower question, namely how far a finite Koopman-style latent operator, trained with local one-step losses and an orthogonality constraint, can carry the low-resolution temporal part of a Kolmogorov-flow AR–SR pipeline before diffusion is used for spatial reconstruction.

Chapter 3

Background

This chapter presents the background required for the thesis. It is divided into three sections, namely the physical system, the autoregressive (AR) module, and the super-resolution (SR) module. The first section explains the physical problem being studied. The second section explains the dynamical-systems foundations of the AR module and time-evolution theory, while the third section explains the probabilistic foundations of the diffusion-based SR module.

Throughout this chapter, the main distinction is that the AR module learns to produce a sequence of vorticity fields by optimizing for an approximately linear transformation in a latent space, whereas the SR module learns to map a low-resolution vorticity field to a high-resolution vorticity field through conditional generation.

3.1 Kolmogorov Flow and Vorticity Fields

The physical problem considered in this thesis is the prediction of two-dimensional Kolmogorov-flow vorticity fields, which provide a canonical forced-fluid setting for studying dynamical behaviour. This problem includes turbulent interactions, and the main focus is therefore on statistically stationary regimes. From a computational perspective, a Kolmogorov-flow trajectory is represented as a sequence of single-channel two-dimensional scalar images, where each frame is a spatial vorticity field or state. Such flows are usually generated by simulators, producing snapshots $\omega(x, y, t)$, where the scalar value at each spatial coordinate describes the local rotation of the flow. A full frame is therefore treated as one state of the dynamical system. In the remaining material, the terms vorticity, spatial field, and state refer mainly to the theoretical perspective, whereas the terms frame, image, and snapshot refer mainly to the computational perspective.

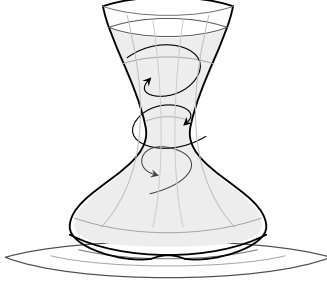


Figure 3.1: A glass of Turkish tea as a simple depiction of vorticity assuming three dimensions of space. The local swirling curls represent vorticities localized on infinitesimal chunks of space within the glass of tea.

For further simplicity, assuming a two-dimensional velocity field $\mathbf{u}(x, y, t) = (u(x, y, t), v(x, y, t))$, incompressibility states

$$\nabla \cdot \mathbf{u} = \partial_x u + \partial_y v = 0,$$

where, for any local infinitesimal square of fluid, the flow is described by two components, that is, a horizontal velocity u and a vertical velocity v . The term $\partial_x u$ asks whether the horizontal velocity u changes when moving horizontally in the x direction, whereas the term $\partial_y v$ asks whether the vertical velocity v changes when moving vertically in the y direction. Together, these partial derivatives cancel, meaning that no fluid is generated or removed from such a square at any point during the time evolution of the dynamical system.

In this thesis, vorticity is the scalar quantity being studied and is defined as

$$\omega(x, y, t) = (\nabla \times \mathbf{u})_z = \partial_x v(x, y, t) - \partial_y u(x, y, t).$$

The following figure depicts a more formal perspective on vorticity fields. For a single vorticity field at time t in continuous time, each local spatial point can be associated with a rotational tendency. For example, if the velocity v increases from left to right, then the partial term $\partial_x v > 0$ contributes positively to $\omega = \partial_x v - \partial_y u$, leading to a counterclockwise local rotation as shown in Figure 3.2.

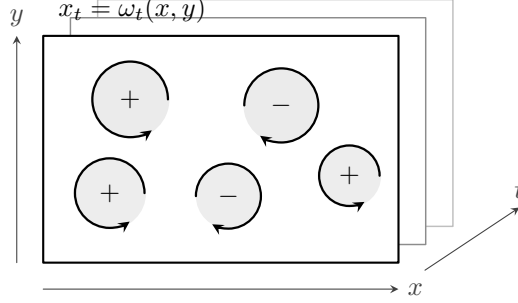


Figure 3.2: Conceptual sketch of a vorticity field, along the time axis. Each field represents the same domain and the front field shows the field $\omega_t(x, y)$ at time t assuming continuous-time physics. The signed vortical patches indicate a local rotation that can take both positive and negative values.

This is why the fields used in this thesis can be treated computationally as single-channel scalar images, as each local grid cell stores a signed amount of local rotation rather than a two-channel velocity vector. In continuous time, these images are samples of the vorticity field $\omega(x, y, t)$. The governing equation describes how this field evolves in continuous time,

$$\partial_t \omega + \mathbf{u} \cdot \nabla \omega = \frac{1}{Re} \Delta \omega - \mu \omega + f_\omega(y),$$

where Re controls viscosity through $\nu = 1/Re$, μ is the linear drag coefficient, and f_ω is the vorticity forcing. As an example of the parameter values used in this thesis, the main dataset uses the configuration

$$Re = 1000, \quad \mu = 0.1, \quad f_\omega(y) = -n_f \cos(n_f y), \quad n_f = 4.$$

As for the governing evolution equation, the dynamics are highly nonlinear and are composed of four interactions. The nonlinear advection term $-\mathbf{u} \cdot \nabla \omega$, when moved to the right-hand side, transports the existing vorticity field by the velocity field $\mathbf{u}$, and it does not create vorticity directly. Instead, it moves the existing field spatially, deforming or evolving it in the process. The viscous diffusion term $\frac{1}{Re} \Delta \omega$ diffuses vorticity, meaning that small local variations are smoothed. The coefficient $\frac{1}{Re}$ adjusts this smoothing effect. Higher Re yields dynamics with finer spatial detail, making the prediction problem usually harder from a machine-learning perspective. The drag term $-\mu \omega$ damps the advected and diffused field, meaning that it pushes values to decay toward zero. Finally, the forcing term is an external input pushed into the field that keeps the structure from dying out. In the Kolmogorov-flow setting, the forcing depends only on the vertical y axis and has a sinusoidal spatial pattern.

Figure 3.3 visualizes these four terms schematically. The box in which the four terms live is the spatial field, and at any coordinate their interaction is compounded, creating a highly nonlinear system. The depiction shows each term in isolation, but it should be acknowledged that their interaction influences every local point within the current field at time t .

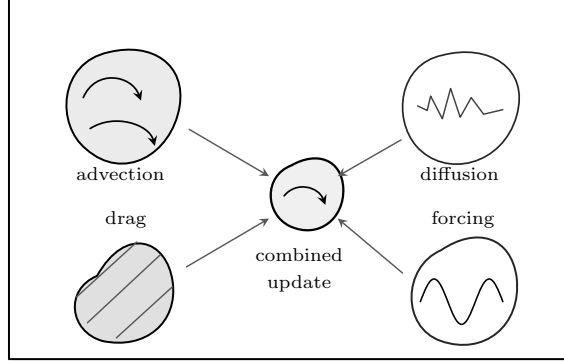


Figure 3.3: Conceptual decomposition of the four terms influencing vorticity field evolution in continuous-time t . The four terms are: advection, diffusion, drag and forcing. The arrows point to the center depicting how they could interact visually to produce the next vorticity state.

To return to the computational perspective, the continuous time variable t is replaced by the discrete index k , which indicates sampled vorticity fields in time and may also be understood as an index along a trajectory tensor. The sampling relation between the continuous-time field and the discrete-time dataset is written as

$$x_k = \omega(\cdot, \cdot, t_k), \quad t_k = t_{\text{start}} + k\Delta t_{\text{save}}.$$

Therefore, physical frames can be sampled in practice by using a simulator to advance the vorticity field in small internal steps, while a recording configuration determines which states are saved. Recording here refers to saving the snapshot of scalar vorticity values to disk as part of the dataset.

For example, in the generated dataset, frames are not saved after every internal simulator step. The internal step size is $\Delta t = 10^{-3}$, while frames are saved every 1000 internal steps. This gives consecutive saved frames separated by $\Delta t_{\text{save}} = 1.0$ simulation time units. Although building and configuring the simulator, often referred to as a numerical solver, is important, the details are not covered in depth in this section. For more details, see Section 4.2.2.

After a numerical solver is run, the dataset can be formed with M unique trajectories, where each trajectory refers to a sequence of recorded vorticity snapshots. Each trajectory has $T + 1$ frames, and the raw spatial resolution N_{HR} can be configured beforehand through the numerical solver parameters. A dataset can be represented as

$$\mathcal{D}^{HR} = \left\{ x_k^{HR,(m)} \right\}_{m=1,k=0}^{M,T}, \quad x_k^{HR,(m)} \in \mathbb{R}^{N_{HR} \times N_{HR}},$$

where each element $x_k^{HR,(m)}$ is one discretized vorticity field. Depending on the initialization of a numerical solver, early frames may belong to a transient or warm-up period, where the flow still carries the influence of the initial condition. In this thesis, the majority of the final experiments are concerned with the post-warm-up period, where fields have converged to a statistically stationary regime, often referred to here as the chaotic regime.

So far, the physical system originating from continuous-time physics has been reduced to a

discrete-time ordered sequence of vorticity fields to form a dataset. From a typical supervised machine-learning perspective, the next question becomes how this computationally expensive task of numerical solvers can be approximated to a plausible degree, where a learned model predicts frames forward in time in an autoregressive manner. Therefore, the next section begins by discussing the mathematical language used to design the physics-inspired autoregressive architecture.

3.2 Autoregressive Module

The autoregressive (AR) module is concerned with predicting frames forward in time, and the particular architecture design it uses from a machine-learning perspective is derived from physics, namely Koopman theory. Therefore, this section begins by defining the mathematical fundamentals, such as finite-dimensional vector spaces, function spaces, normed spaces, inner-product spaces, and Hilbert spaces, to make the theoretical perspective explicit. Dynamical systems are then introduced again to provide the perspective of Koopman operator theory, connecting the mathematical setting to the physical problem. Finally, the link between the Koopman operator and the autoencoder architecture design is set out to connect this theory to the machine-learning perspective.

The previous section already introduced the continuous-time Kolmogorov-flow evolution dynamics and conceptually explained how sampling from it produces a dataset of vorticity frames. The AR module starts from this discretized viewpoint. If x_k denotes a saved low-resolution vorticity frame, then the unknown evolution can be written conceptually as

$$x_{k+1}^{LR} = S_{\Delta t_{\text{s ave}}} (x_k^{LR}).$$

For additional context, the spatial space in which the AR module operates is a lower-resolution space compared to the numerical solver outputs. The motivation behind this will be linked later with the super-resolution (SR) module in the next section.

The learning problem for the AR module is then to replace this unavailable sampled flow map with a modelled transition that can carry the dynamics forward in time for n steps,

$$\hat{x}_{k+1}^{LR} = F_{\theta}(x_k^{LR}), \quad \hat{x}_{k+n}^{LR} = F_{\theta}^n(x_k^{LR}).$$

The autoregressive nature here refers to the model’s ability to predict from its own outputs, recomputing frames to create a sequence of frames, often referred to as a trajectory or a rollout. Koopman theory enters as one particular way of structuring F_{θ} , where the rollout generator takes a nonlinear frame, encodes it once into a latent representation, advances it by a finite linear operator, and decodes each latent output independently back to a vorticity frame.

To understand the AR module and its experiments, only a small amount of functional analysis is required. Returning to the continuous-time physics, the vorticity fields must be treated as mathematical objects that can be compared, added, and evolved. For this language, norms are needed for defining vector spaces, as well as for defining the error space of optimization problems that will be used when the machine-learning perspective arrives. Inner products are needed because orthogonality and latent-space geometry appear in the AR experiments. The physics perspective

uses Hilbert-space language, which provides a clean setting for these ideas. However, the practical object remains simple, that is, a vorticity frame is handled as one state, and a model prediction is judged by how well it behaves as a future state.

The transition from physics to the machine-learning perspective is important because, although the physical state is a vorticity field with continuous field, the neural model sees a discretized array. Thus, any low-resolution vorticity frame can be represented as

$$x_k^{LR} = \text{vec}(\omega_k(x_i, y_j))_{i,j=1}^{N_{LR}} \in \mathbb{R}^{N_{LR}^2},$$

where N_{LR}^2 is the number of total pixels. This is the finite-dimensional setting that the AR module encodes, advances, and decodes. The Hilbert-space language will be required when explaining function spaces. Here, the flattening notation is only useful mathematically, since the neural model itself preserves the array structure provided as input.

From Finite-Dimensional Vectors to Function Spaces

First, recall the finite-dimensional linear algebra setting, where vectors are elements of a vector space and may be represented, after choosing a basis, by finite lists of numbers. For example, a vector in $\mathbb{R}^3$ may be written as (a_1, a_2, a_3) .

With such vectors, one can measure properties such as length, and relations such as distance and orthogonality, which are some of the fundamental properties in linear algebra.

A typical vorticity field is infinite-dimensional because it belongs to a space of functions, while the neural model can only operate on samples from discretized-time frames and discretized spatial dimensions. For example, in a discretized vorticity frame, a scalar value is assigned to each coordinate or index $(x, y) \in D$ in the spatial domain. Evaluating the field at that coordinate returns a value, for example $u(x, y)$ for a velocity component. In this thesis, as explained previously, the quantity of interest is vorticity, denoted by $\omega : D \rightarrow \mathbb{R}$. The function ω represents the field in an infinite-dimensional space, and such fields are referred to as states in continuous time t . Therefore, a state may be written as $x = \omega$.

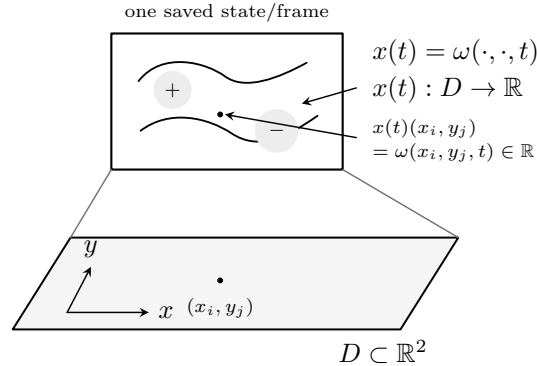


Figure 3.4: Conceptual depiction of a vorticity field at time t along continuous-time dynamics. The domain $D \subset \mathbb{R}^2$ contains spatial coordinates, while the full state $x(t) = \omega(\cdot, \cdot, t)$ assigns one scalar value to each point, with $x(t)(x_i, y_j) \in \mathbb{R}$. The edge lines indicate the whole-domain to whole-field relation, while the marked point shows one pointwise evaluation.

From the physics perspective, a vorticity field is not a finite-dimensional vector, but a function defined over the whole spatial domain. Moving further into the function-space viewpoint requires a way to measure the size of a field and the distance between fields, which leads to the next subsection on normed and inner-product spaces.

Normed and Inner Product Spaces

In the finite-dimensional setting, a normed space is a vector space equipped with a rule for measuring the size of its elements. This rule is called the norm and is denoted by $\|v\|$. The norm must be nonnegative, vanish only at the zero vector, scale correctly under scalar multiplication, and satisfy the triangle inequality. Once a norm is given, it also defines a distance by

$$d(v, w) = \|v - w\|.$$

For example, if $v = (v_1, v_2, \dots, v_n)$, then its usual Euclidean norm is

$$\|v\| = \sqrt{v_1^2 + \dots + v_n^2}.$$

In the infinite-dimensional setting of function spaces, an important example is the L^2 norm,

$$\|\omega\|_{L^2(D)} = \left(\int_D |\omega(x, y)|^2 dx dy \right)^{1/2}.$$

This norm measures the overall magnitude of a function over the domain and defines a natural distance between two fields.

A normed space measures size and distance. An inner-product space has more structure, because the inner product also measures geometric relations between elements. In the finite-dimensional setting, an inner product induces notions of length, angle, orthogonality, and projection. The standard example is the dot product,

$$v \cdot w = v_1 w_1 + \dots + v_n w_n.$$

In the infinite-dimensional setting, for vorticity fields, a standard inner product is

$$\langle \omega_1, \omega_2 \rangle = \int_D \omega_1(x, y) \overline{\omega_2(x, y)} dx dy.$$

This quantity measures the overlap between two functions. For example, if $\langle \omega_1, \omega_2 \rangle = 0$, then the two fields are said to be orthogonal in the same geometric sense that perpendicular vectors are orthogonal in the finite-dimensional setting.

Norms and inner products therefore translate the familiar geometric tools of finite-dimensional linear algebra into the infinite-dimensional setting. They describe how large functions are, how close they are, and whether they point in orthogonal directions. However, one further property is needed, namely Cauchy sequences of functions should converge to functions that remain within the space in which they are already being studied. This property leads to the idea of completeness and Hilbert spaces.

Hilbert Spaces

Formally, a Hilbert space $\mathcal{H}$ is a complete inner-product vector space. It includes both finite-dimensional and infinite-dimensional settings, while preserving the geometric ideas introduced above. Because Hilbert spaces are complete, limits of Cauchy sequences remain inside the space. A Cauchy sequence is a sequence whose elements become arbitrarily close to one another with respect to the Hilbert-space norm. This property is essential because it ensures that limiting processes, which occur frequently in analysis and partial differential equations (PDEs), do not lead outside the space being studied. Thus, Hilbert spaces provide a well-behaved setting for studying dynamical systems.

This Hilbert-space setting also supports the Koopman perspective introduced later in this section. In the physical setting, a vorticity state may belong to a space such as $L^2(D)$. In the Koopman setting, however, the operator acts on observables, which are functions of the state. These observables may be studied in a space such as $L^2(\mathcal{X}, \mu)$, where $\mathcal{X}$ denotes the state space and μ is a measure on that space. The distinction between vorticity fields and observables will be made clear later.

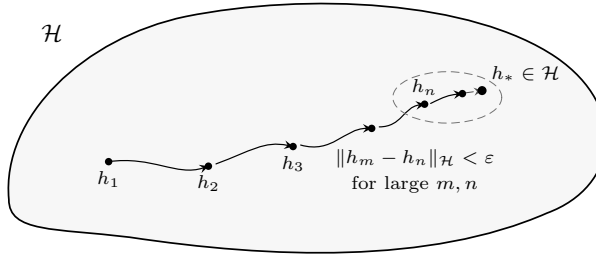


Figure 3.5: Conceptual depiction of Cauchy sequences in a complete Hilbert space. The sequence elements, which may be functions such as vorticity fields, become arbitrarily close to one another, and the limiting element remains inside $\mathcal{H}$.

From the perspective of physics and fluid dynamics, each state may be treated as a vorticity field $\omega(\cdot, \cdot, t)$ at continuous time t , meaning that $\omega(\cdot, \cdot, t) \in L^2(D)$. Therefore, a sequence of increasingly similar vorticity states may be viewed as a sequence of points approaching a limiting state in the same space. In Figure 3.5, if each function h_n is regarded as a vorticity field and the sequence is regarded as an evolution toward h_* , then the underlying physical dynamics may still be highly nonlinear. Therefore, the next question becomes how one state changes into another with time, moving the discussion from function spaces to dynamical systems and their evolution operators.

Dynamical Systems, Flows, and Observables

Having introduced a space in which fluid states can be treated as geometric objects, we now consider how these states change over time. A dynamical system evolves in time, and this evolution may be nonlinear. Let $\mathcal{X}$ denote the state space and let $x(t) \in \mathcal{X}$ denote the state of the system

at a given time t . The flow map S_t represents the evolution of these states and is defined as

$$S_t : \mathcal{X} \rightarrow \mathcal{X}.$$

For example, given an initial state $x(0)$, the evolved state after time t is $S_t(x(0))$, which yields the state $x(t)$. For a deterministic system such as fluid dynamics, the initial state determines its future trajectory through the state space. At $t = 0$, no evolution has taken place, so S_0 is the identity mapping.

For any such system, the flow also satisfies

$$S_{t+s} = S_t \circ S_s,$$

which means that the flow is compositional. Rather than only studying the field state itself, one is often interested in a particular quantity associated with it. Such a quantity is described by a different type of function f that acts upon the whole field, namely an observable,

$$f : \mathcal{X} \rightarrow \mathbb{R} \quad \text{or} \quad f : \mathcal{X} \rightarrow \mathbb{C}.$$

An observable takes a complete field $x \in \mathcal{X}$ and returns a scalar value. For example, such a quantity can be the vorticity at a fixed spatial location, the total kinetic energy, or the enstrophy.

If these observables are square-integrable with respect to a measure μ defined on the state space, they belong to the Hilbert space $L^2(\mathcal{X}, \mu)$. This should be distinguished from a space such as $L^2(D)$, whose elements may represent the fluid states themselves as functions over the physical domain.

This provides the connection between the dynamical-systems perspective and the Hilbert spaces introduced previously. The flow S_t evolves the vorticity states $x = \omega$ nonlinearly, while observables f evaluate quantities of those states. The next question then becomes how observables themselves evolve under the flow. This is where Koopman operator theory becomes relevant, namely that observables may evolve linearly even when the state dynamics underneath are nonlinear.

Although the next section is concerned with Koopman operators directly, it is helpful to state the machine-learning connection in advance. From this perspective, the latent space learned by the AR module should behave similarly to a finite space of observables. If such a space can be learned, then linear propagation of the encoded state using a finite Koopman-style operator becomes possible. The nonlinear evolution is difficult in pixel space, but it may be easier to approximate as linear evolution after encoding the vorticity fields.

Linear Operators on Hilbert Spaces

Previously, observables f were explained as functions belonging to a different Hilbert space. They map vorticity fields to scalar values, whereas the vorticity fields themselves, written as $x = \omega$, are functions over the spatial domain. Therefore, rather than studying only the nonlinear evolution S_t of individual vorticity fields, one can study the induced evolution $\mathcal{K}_t$ of observables, where $\mathcal{K}_t$ is the Koopman operator. In the later machine-learning setting, K is reserved for the learned finite matrix.

Linearity

The Koopman operator is defined by

$$(\mathcal{K}_t f)(x) = f(S_t(x)).$$

Here, the operator $\mathcal{K}_t$ acts on the observable f , which is evaluated at the vorticity state x . This is equivalent to evaluating the evolved state $S_t(x)$ with the same observable. The flow S_t may be nonlinear, but the corresponding Koopman operator is linear. To show the linearity, suppose there are two observables f and g , with scalars a and b ,

$$\begin{aligned} \mathcal{K}_t(af + bg)(x) &= (af + bg)(S_t(x)) \\ &= af(S_t(x)) + bg(S_t(x)) \\ &= a(\mathcal{K}_t f)(x) + b(\mathcal{K}_t g)(x). \end{aligned}$$

This means

$$\mathcal{K}_t(af + bg) = a\mathcal{K}_t f + b\mathcal{K}_t g.$$

Thus, although states may evolve nonlinearly, $\mathcal{K}_t$ is linear with respect to observables. If two observables are combined as $af + bg$, then applying the Koopman operator preserves this combination. This also holds for any finite linear combination of observables.

However, from the machine-learning perspective in the finite-dimensional setting, the operator to be learned becomes a square matrix, and the observables are represented by the latent coordinate system on which the matrix performs a linear transformation. Therefore, the AR module requires learning both a meaningful latent representation of observables and the linear transformation matrix.

Such a meaningful coordinate system required by the AR module can be defined as

$$z_k = E_\theta(x_k), \quad z_k = \begin{bmatrix} g_1(x_k) & g_2(x_k) & \cdots & g_d(x_k) \end{bmatrix}^\top,$$

where the encoder E_θ learns d observable-like coordinates $g_1, \dots, g_d$, corresponding to the dimensionality of the latent vector.

The matrix form of the Koopman approximation is then written as a linear transformation,

$$z_{k+1} \approx K z_k, \quad K \in \mathbb{R}^{d \times d}.$$

This is the modelling assumption later stressed by the strict latent rollout experiments for the AR module, where powers of K allow n -step saved-frame rollouts without requiring the state to be encoded again.

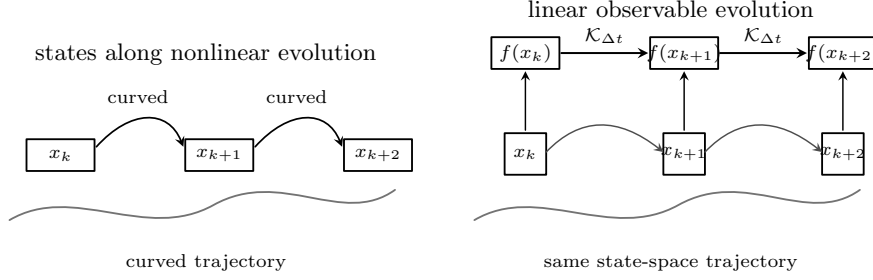


Figure 3.6: Conceptual sketch of Koopman linearity. The left panel shows nonlinear evolution of vorticity states depicted by curved arrows. The right panel keeps the same curved state trajectory underneath, while the observable evaluations above it are advanced linearly by the Koopman operator $\mathcal{K}_{\Delta t}$.

In the finite-dimensional setting of the AR module, the learned evolution is represented by a matrix acting on a latent vector. In the theoretical Koopman setting, however, the operator acts on a function space of observables and is therefore generally infinite-dimensional. This creates the link between the two viewpoints, where the learned matrix is a finite approximation of an operator whose behaviour can be studied through spectral quantities.

Just as a matrix can be studied through its eigenvalues and eigenvectors, a Koopman operator can be studied through eigenvalues and eigenfunctions. This is important for the AR module because rollout is performed by repeatedly applying the learned matrix, so its spectrum gives information about which latent directions may decay, grow, or oscillate during prediction. The next subsection therefore introduces Koopman eigenfunctions, which are special observables whose evolution takes a particularly simple spectral form.

Koopman Eigenfunctions

A Koopman eigenfunction is a special type of observable that behaves like an eigenvector does for a matrix. In the finite-dimensional setting, as a reminder, an eigenvector satisfies

$$A\mathbf{v} = \lambda\mathbf{v},$$

which means that applying the matrix changes the size or phase of the vector while keeping its direction fixed. For the Koopman operator, the corresponding object is not a vector in D , but an observable ϕ defined on the Hilbert space $L^2(\mathcal{X}, \mu)$. The eigenvector idea translates as

$$\mathcal{K}_t\phi = e^{\nu t}\phi.$$

This also implies

$$e^{\nu t}\phi(x) = \phi(S_t(x)),$$

where the form is similar to $\mathcal{K}_t f(x) = f(S_t(x))$.

Here, although the vorticity field x is advanced by S_t , the special observable ϕ changes only through the exponential coefficient $e^{\nu t}$. The term ν determines whether this observable component decays, grows, or oscillates.

In the discrete setting, one saved step corresponds to the time interval Δt_{save} . Therefore, the continuous-time factor $e^{\nu_i t}$ is related to a one-step discrete multiplier of the learned matrix by

$$\lambda_i \approx e^{\nu_i \Delta t_{\text{save}}}.$$

This connection between the theoretical and practical side is useful because it gives a conceptual idea of what is happening, that is, the AR matrix has discrete eigenvalues λ_i , and repeated rollout applies these factors through powers of the learned matrix.

Returning to the infinite-dimensional setting, a more general observable may be represented as a combination of Koopman eigenfunctions,

$$f = \sum_i c_i \phi_i.$$

Applying the Koopman operator then gives

$$\mathcal{K}_t f = \sum_i c_i e^{\nu_i t} \phi_i.$$

Here, the concept to remember is that observables can be decomposed into simpler spectral pieces, and each piece keeps its form fixed while only its coefficient changes under the evolution. The AR module uses this as motivation, but the learned latent coordinates are not guaranteed to be exact Koopman eigenfunctions. They are instead treated as general observable-like coordinates. So far, single-step dynamics have motivated the usefulness of observables, but under long and repeated rollouts another perspective is needed to summarize the stability problem.

Koopman-Invariant Subspace

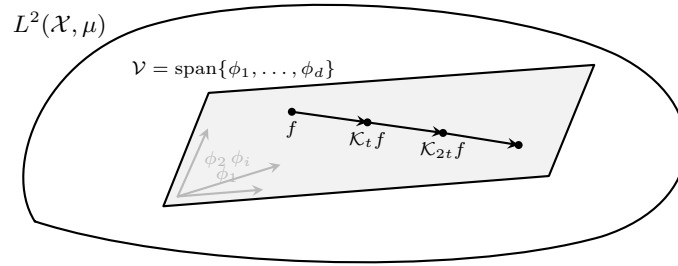


Figure 3.7: Conceptual sketch of a Koopman-invariant observable subspace. The full Koopman operator acts on the observable space, but observables inside the finite span of eigenfunctions remain inside the same subspace under repeated evolution. Each eigenfunction direction changes only by its corresponding spectral multiplier.

Returning to the theoretical perspective, let $\mathcal{V}$ be the subspace spanned by a finite collection of Koopman eigenfunctions,

$$\mathcal{V} = \text{span}\{\phi_1, \dots, \phi_d\}.$$

The subspace $\mathcal{V}$ is Koopman-invariant if

$$\mathcal{K}_t(\mathcal{V}) \subseteq \mathcal{V}.$$

This means that any observable inside the span of $\phi_1, \dots, \phi_d$ remains representable after transformation by the Koopman operator. Keeping the coordinate space unchanged under the operator is a useful property because, when this is satisfied, the infinite-dimensional Koopman operator can be represented by a finite-dimensional matrix on that same subspace.

In the finite-dimensional machine-learning setting, the coordinate system is again a learned latent vector. If z_k belongs to such a finite coordinate space, then repeated latent evolution is written as

$$z_{k+n} \approx K^n z_k.$$

If the learned matrix is diagonalizable,

$$K = V\Lambda V^{-1}, \quad K^n z_k = V\Lambda^n V^{-1} z_k$$

then, the behaviour of long rollouts is controlled by the eigenvalues inside Λ^n . Directions associated with $|\lambda_i| > 1$ may grow, directions with $|\lambda_i| < 1$ may decay, and complex eigenvalues may describe oscillatory latent behaviour. This is the practical reason that spectrum, eigenfunctions, and invariant subspaces are introduced before connecting Koopman theory to the autoencoder architecture.

Connection to Koopman Autoencoders

To finally connect this theory to the practical setting, suppose an encoder extracts a latent representation of the provided vorticity frame,

$$z_k = \Phi(x_k),$$

where the latent representation z_k may be interpreted as learned observables of the state x_k , and not necessarily as an exact coordinate system of eigenfunctions.

A finite-dimensional latent dynamics matrix K then attempts to approximate the linear transformation over the discretized observable space over one saved discrete step,

$$z_{k+1} = K z_k.$$

Ideally, the goal is to learn an encoder that can extract useful observables from a given vorticity frame, where these observables evolve approximately linearly and span an approximately Koopman-invariant subspace under repeated transformation.

For a saved time step Δt_{save} , repeated latent transformation aims to satisfy

$$K^n \Phi(x_k) \approx \Phi(S_{n\Delta t_{\text{save}}}(x_k)),$$

where a decoder Ψ then maps the transformed latent state back to the physical vorticity frame space,

$$\Psi(K^n \Phi(x_k)) \approx S_{n\Delta t_{\text{save}}}(x_k).$$

The nonlinear dynamics are therefore represented through an encoder and decoder, which act as tools for extracting an approximately Koopman-invariant subspace of observables. In the ideal case, these observables would correspond to eigenfunctions, while the temporal transformation between latent states is approximated with a square matrix.

3.3 Super-Resolution Module

This section begins with the probabilistic background, latent variables, and the evidence lower bound (ELBO) objective, in order to prepare the mathematical background useful for understanding the intuition of the methodology used here. These concepts are then connected to the machine-learning perspective. In the process, this section touches on variational autoencoders (VAEs) as a stepping stone for understanding how the optimized objective, that is, the ELBO, translates to generative modelling.

Previously, the AR module set the task of rolling out vorticity frames in pixel space by learning a finite-parameter linear transformation that propagates learned latent coordinates forward in time. The SR module is different because it operates on a given single time step, unlike the AR module, which requires a pair of states. The AR module has already been described from the discrete viewpoint. Different from the AR section, this section introduces the notion of resolution. Therefore, assume that the AR section operates on low-resolution vorticity frames denoted by x_k^{LR} , while x_k^{HR} denotes the corresponding high-resolution frame on which the SR module operates.

For the remaining context, this section assumes the discretized space of vorticity frames using the index k for physical time. The diffusion process uses a separate artificial index τ , with $\tau_{\max}$ denoting the final or noisiest diffusion step. The unknown up-resolution task can then be written as

$$x_k^{HR} \sim p(x^{HR} | c_k), \quad c_k = U(x_k^{LR}).$$

Here, c_k is the upsampled low-resolution condition containing coarse details, and U is the upsampling operator that maps the low-resolution frame to the high-resolution grid. The goal of the SR module is therefore to recover the details missing from the coarse upsampling process.

Formally, the learning problem becomes finding a map from the coarse frame with missing vortical detail to an enhanced high-resolution frame at a given physical time index k ,

$$\hat{x}_k^{HR} \sim p_\phi(x^{HR} | c_k), \quad \hat{\epsilon} = \epsilon_\phi(x_{k,\tau}, \tau, c_k).$$

Here, the left expression defines the super-resolution task as a probabilistic conditional generative model, whereas the right expression defines the specific diffusion objective used by the generative model. The learning task is to predict the noise in a corrupted high-resolution sample $x_{k,\tau}$ while being guided by the upsampled coarse condition c_k .

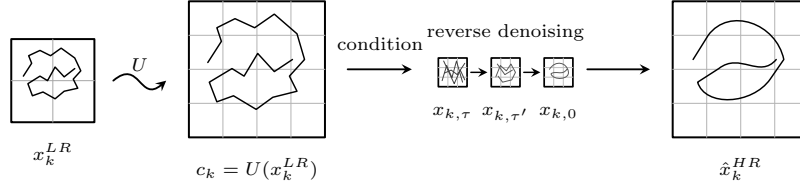


Figure 3.8: Conceptual sketch of the SR module. The low-resolution frame is upsampled to form the condition c_k . Starting from a noisy high-resolution sample $x_{k,\tau}$, the diffusion sampler iteratively removes predicted noise through intermediate diffusion states such as $x_{k,\tau'}$, while using the condition, producing the high-resolution reconstruction $\hat{x}_k^{HR}$.

From Figure 3.8, it can be seen that for each frame from the AR module, the SR module can be applied without requiring knowledge of the previous frames. Later, the joint pipeline will be defined, where the AR and SR modules are combined to predict a low-resolution rollout first, and then the SR module reconstructs the corresponding high-resolution field frame-wise,

$$\hat{x}_{k+n}^{HR} \sim p_\phi(x^{HR} | U(\hat{x}_{k+n}^{LR})).$$

This is the point where the probabilistic language becomes necessary. Upsampling alone gives a deterministic coarse image c_k , but it does not define the missing fine-scale vorticity. Many high-resolution fields may be compatible with the same low-resolution condition. Therefore, the SR problem is better described as a conditional distribution rather than as a single fixed output. The model is asked to assign probability to plausible high-resolution fields given the observed condition.

The next subsection introduces the terms needed for that viewpoint. The observed variable is the data sample we want to explain, such as the clean high-resolution field x_k^{HR} . A latent variable is an unobserved quantity used by the model to explain or generate that sample. A joint model describes the probability of both the observed and latent quantities together, while a posterior asks which latent quantities are likely after the observation is known. When this posterior cannot be computed directly, an approximate posterior or proposal distribution is introduced. This is the role played by variational inference and the ELBO.

In diffusion models, these latent variables are not only a single compressed code. They can instead be read as noisy intermediate states between a clean field and pure Gaussian noise. In the SR setting, these states may be written as $x_{k,\tau}$. This is why the discussion moves from $p_\phi(x^{HR} | c_k)$ to marginal likelihood, posteriors, joint models, and ELBOs, which explain how a denoising model can be trained as a probabilistic generative model.

Latent-Variable Generative View

First, recall the generative model language, where the observed sample, or rather the vorticity frame x , is often assumed to depend on unobserved variables z , also called latent variables. These latent variables are similar in name to the latent variables of the AR encoder, but they do not require the same physics interpretation. The marginal likelihood of the sample $p_\theta(x)$ is typically

obtained by integrating out those latent variables,

$$p_\theta(x) = \int p_\theta(x, z) \mathrm{d}z.$$

From the perspective of the SR module, the conditional task is

$$p_\theta(x_k^{HR} | c_k) = \int p_\theta(x_k^{HR}, z | c_k) \mathrm{d}z,$$

where the condition is the usual upsampled low-resolution guidance for the model. As stated previously, this is impractical because it requires integrating over an unknown latent space. In diffusion models, z will later be identified not with a single compressed code, but with a noisy latent chain $x_{1:\tau_{\max}}$, while the reverse model learns how to move from noise back toward the clean state x_0 .

Returning to the theoretical perspective, one possible workaround is to consider the posterior distribution. In the conditional SR setting, this can be written as

$$p_\theta(z | x_k^{HR}, c_k) = \frac{p_\theta(x_k^{HR}, z | c_k)}{p_\theta(x_k^{HR} | c_k)}.$$

However, this expression still requires the marginal likelihood $p_\theta(x_k^{HR} | c_k)$, which is impractical because the full distribution of vorticity frames is unknown.

As a possible solution, variational inference introduces an approximate posterior $q_\phi(z | x)$, with parameters ϕ . The objective then becomes maximizing a lower bound on the log likelihood,

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right].$$

This is the evidence lower bound, namely the ELBO, and maximizing it becomes the objective. It can be understood as trading an intractable marginal likelihood for an expectation under a tractable proposal distribution. The gap between the log likelihood and the ELBO is a KL divergence between the approximate posterior and the true posterior, so maximizing the ELBO encourages the approximation to match the latent structure implied by the data.

This idea appears in variational autoencoders, but it is not limited to them. For this thesis, the important part is the general pattern

$$\mathbb{E}_{\text{proposal}} \left[\log \frac{\text{joint model}}{\text{proposal distribution}} \right].$$

Diffusion models can also be viewed through this variational latent-variable lens, where the proposal becomes the known noising process. This noising process is the previously mentioned chain of noisy frames and is usually referred to as the forward corruption process. The learned model therefore becomes the reverse denoising process.

For the SR module used here, it is useful to separate training from the sampling process. During training, the noisy frame is made by adding Gaussian noise to the clean high-resolution target,

$$x_{k,\tau} = \sqrt{\bar{\alpha}_\tau} x_k^{HR} + \sqrt{1 - \bar{\alpha}_\tau} \epsilon,$$

and the model predicts ϵ from $(x_{k,\tau}, \tau, c_k)$. During inference, the clean high-resolution target is not available, so the reverse DDIM sampler starts from $x_{k,\tau_{\max}} \sim \mathcal{N}(0, I)$ and is guided by the condition c_k .

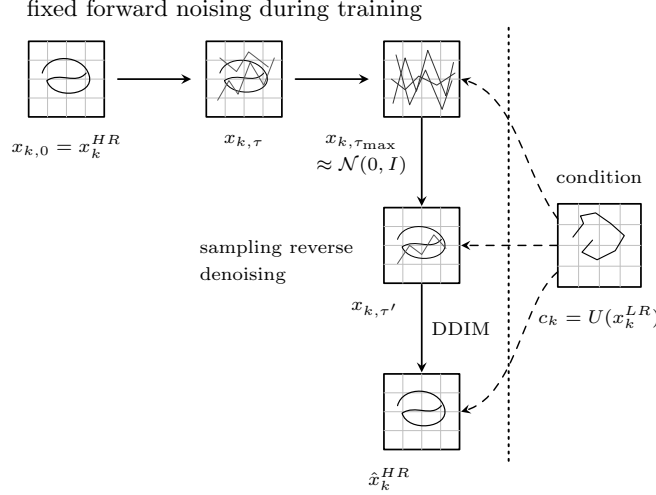


Figure 3.9: Conceptual sketch of the diffusion noising and denoising chain used for the SR module. The upper row shows the training construction, where the clean high-resolution target $x_{k,0} = x_k^{HR}$ is gradually noised into intermediate noisy states $x_{k,\tau}$ and then toward the final noisy state $x_{k,\tau_{\max}}$. At sampling time, the reverse chain is initialized from a near-Gaussian state $x_{k,\tau_{\max}}$ and denoised through selected DDIM steps while being conditioned on the upsampled low-resolution frame c_k .

Diffusion as a Latent Chain

In a diffusion model, the observed clean frame is written as x_0 , while $x_1, \dots, x_{\tau_{\max}}$ are noisy intermediate states that can be produced by the forward process. The previously mentioned forward process gradually corrupts the frame,

$$q(x_{1:\tau_{\max}} | x_0) = \prod_{\tau=1}^{\tau_{\max}} q(x_{\tau} | x_{\tau-1}),$$

where the learned model attempts to reverse this chain,

$$p_{\theta}(x_{0:\tau_{\max}}) = p(x_{\tau_{\max}}) \prod_{\tau=1}^{\tau_{\max}} p_{\theta}(x_{\tau-1} | x_{\tau}).$$

The marginal likelihood of a clean sample is therefore

$$\log p_{\theta}(x_0) = \log \int p_{\theta}(x_{0:\tau_{\max}}) dx_{1:\tau_{\max}},$$

which has the same intractable structure as the latent-variable model above because it would need to consider every possible noisy path. Instead of maximizing this difficult likelihood directly, diffusion uses the known forward noising process as the proposal distribution, giving the diffusion

ELBO,

$$\log p_\theta(x_0) \geq \mathbb{E}_{q(x_{1:\tau_{\max}}|x_0)} \left[\log \frac{p_\theta(x_{0:\tau_{\max}})}{q(x_{1:\tau_{\max}}|x_0)} \right].$$

This bound decomposes into terms with interpretable roles, namely a reconstruction term near x_0 , a prior-matching term that pushes the final noisy state toward a simple prior such as a standard Gaussian, and intermediate terms that train the reverse transitions to match the corresponding denoising posteriors. Thus, the theoretical diffusion model is a probabilistic model over an entire noise trajectory.

The important point here is not the exact decomposition of every ELBO term, but the role it gives to diffusion training. The bound turns generation into a sequence of denoising problems, that is, the model starts from a noisy state and learns reverse transitions that gradually denoise frames.

From ELBO to Denoising Objective

Denoising diffusion probabilistic models make this forward noising process practical by choosing a fixed Gaussian distribution [Ho et al., 2020]. For a sampled diffusion step τ , the noisy state can be written directly as

$$x_\tau = \sqrt{\bar{\alpha}_\tau} x_0 + \sqrt{1 - \bar{\alpha}_\tau} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

where $\mathcal{N}(0, I)$ is the Gaussian noise distribution, and $\bar{\alpha}_\tau$ controls how much of the clean frame is retained at diffusion step τ . The diffusion horizon $\tau_{\max}$ is a configurable parameter that defines how many progressively noisier versions of a given frame are used. Rather than learning the full reverse map directly, a neural model is trained to predict the injected noise,

$$\hat{\epsilon} = \epsilon_\phi(x_\tau, \tau).$$

With this parameterization, the variational training problem can be optimized through the following denoising objective,

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}_{x_0, \tau, \epsilon} \left[\|\epsilon_\phi(x_\tau, \tau) - \epsilon\|_2^2 \right].$$

This is the machine-learning perspective of the probabilistic theory, where the model learns the reverse diffusion process by repeatedly solving supervised noise-prediction problems at randomly selected noise levels.

Conditional Diffusion for Super-Resolution

Returning to the perspective of the SR module, it seeks to solve a conditional objective instead, where the model is not asked to generate arbitrary high-resolution vorticity frames, but to sample reconstructed high-resolution frames from existing coarse frames. In this thesis, the clean data is the high-resolution vorticity frame,

$$x_{k,0} = x_k^{HR},$$

and the condition is the upsampled low-resolution field,

$$c_k = U(x_k^{LR}).$$

Then the learned reverse process becomes conditioned on c_k ,

$$p_\phi(x_{k,0:\tau_{\max}} \mid c_k) = p(x_{k,\tau_{\max}}) \prod_{\tau=1}^{\tau_{\max}} p_\phi(x_{k,\tau-1} \mid x_{k,\tau}, c_k).$$

For the specific conditional reconstruction design, this thesis uses the SR3-style formulation [Saharia et al., 2023], where the denoiser receives the noisy high-resolution field, the diffusion step, and the upsampled low-resolution condition,

$$\hat{\epsilon} = \epsilon_\phi(x_{k,\tau}, \tau, c_k),$$

and is trained with the conditional noise-prediction loss,

$$\mathcal{L}_{\text{SR}} = \mathbb{E}_{x_k^{HR}, c_k, \tau, \epsilon} \left[\|\epsilon_\phi(x_{k,\tau}, \tau, c_k) - \epsilon\|_2^2 \right].$$

This gives the conceptual link used in the methodology. The low-resolution condition preserves the coarse frame after being upsampled to the high-resolution grid, while the diffusion model learns how to restore plausible fine-scale vorticity structure by reversing a noising process on the high-resolution field. During inference, the model starts from Gaussian noise $x_{k,\tau_{\max}}$ and repeatedly denoises while conditioning on c_k . In this thesis, that reverse process is sampled with a DDIM-style update [Song et al., 2021] and is used mainly as a practical sampling method, rather than as a separate modelling assumption.

Chapter 4

Method

This chapter presents the methodology and is divided into five main sections, namely the problem setup, the dataset, the autoregressive (AR) module, the super-resolution (SR) module, and the joint module. The first section defines the prediction task independently of the modules developed in this thesis. The second section describes the dataset details and the main alternative dataset explored during the project. The third section presents the AR module and its main design choices, while the fourth section describes the SR module. The fifth section then explains how the AR and SR modules are combined in the joint pipeline. Together, these sections provide the setup required for the later experiments and results.

4.1 Problem Setup

The problem considered in this thesis is the prediction of two-dimensional vorticity fields forward in time. Each state is treated as a single-channel field sampled on a finite grid. Let x_k^{HR} denote the high-resolution vorticity field at discrete time k , and let x_k^{LR} denote its low-resolution counterpart.

The high-resolution field may represent an image of height 256, width 256 and one channel, while low-resolution may represent an image of height 64, width 64 and one channel. The two are related by a downsampling operator D ,

$$x_k^{LR} = D(x_k^{HR}).$$

The AR module predicts future low-resolution states,

$$x_k^{LR} \mapsto x_{k+1}^{LR},$$

whereas the SR module maps low-resolution states back to high resolution fields,

$$x_k^{LR} \mapsto x_k^{HR}.$$

The joint problem is therefore to apply AR prediction forward in time to produce a low-resolution rollout,

$$x_k^{LR}, x_{k+1}^{LR}, \dots, x_{k+n}^{LR},$$

and then reconstruct the rollout at higher resolution,

$$\hat{x}_k^{HR}, \hat{x}_{k+1}^{HR}, \dots, \hat{x}_{k+n}^{HR}.$$

This setup splits the problem into two main subcomponents. The first carries out the rollout forward in time, whereas the second carries out the reconstruction process from low-resolution to high-resolution frames. This is a modular formulation, meaning that the temporal prediction and spatial reconstruction stages can be developed and evaluated separately [Xue et al., 2026].

This naturally leads to the discussion of the dataset and how it is represented to train the modules.

4.2 Dataset

The modules in this thesis require trajectories of vorticity fields sampled on a fixed spatial grid. Each trajectory is a sequence $(x_0, x_1, \dots, x_T)$, where each x_k is a two-dimensional vorticity field snapshot. Since the aim is to study rollout behaviour in a chaotic regime, the dataset should contain post-transient dynamics rather than only the early adjustment from an initial condition. It should also contain enough trajectories so that the model is not evaluated only on a small number of independent flows.

4.2.1 Initial Dataset

The first dataset used in the project was the 40-trajectory dataset associated with [Shu et al., 2023]. This dataset was useful as an initial point because it provided vorticity trajectories at the target spatial resolution, that is, 256×256 , and allowed the AR and SR code paths to be tested early. However, the small number of trajectories, with total count 40, and the presence of transient-state behaviour made it difficult to separate model failure from data limitations. For this reason, the initial dataset is treated as an early development dataset rather than the main dataset for the final experiments.

Some reported experiments in Chapter 5 are based on this dataset and are specified as such where relevant.

4.2.2 Generated Dataset

The main dataset used later in this thesis was generated using the Kolmogorov flow solver associated with [Li et al., 2024]. Candidate trajectories were first evolved through a warm-up period before recording, so that the saved frames were closer to the statistically developed regime of the flow. The generation process used a pseudo-spectral solver with Crank–Nicolson and Heun time stepping, dealiasing, Kolmogorov forcing, and linear drag. From the generated candidate trajectories, non-finite trajectories were removed before training and evaluation. The target was to generate 400 clean trajectories, of which 86 were removed due to unstable simulations.

The generation procedure can be summarized as follows.

Algorithm 1 Kolmogorov flow dataset generation

Require: Reynolds number Re , resolution N , time step Δt , forcing mode n_f , drag μ

Require: GRF parameters (α, τ) , warm-up time T_w , frame count F , record interval s

Require: total trajectories M , batch size B , random seed σ

Ensure: Dataset tensor $D \in \mathbb{R}^{M \times F \times N \times N}$ of vorticity snapshots

Initialise pseudo-spectral solver $\mathcal{S}$ with $(N, Re, \Delta t, n_f, \mu)$

Precompute wavenumber grids, inverse Laplacian, dealiasing mask and GRF spectrum

$D \leftarrow \text{zeros}(M, F, N, N)$

for $b = 0, B, 2B, \dots, M - B$ **do**

 seeds $\leftarrow [\sigma + b, \sigma + b + 1, \dots, \sigma + b + B - 1]$

$\hat{\omega}_0 \leftarrow \text{sample_GRF}(\text{seeds}, \alpha, \tau)$

 Set solver state $\mathcal{S}.\hat{\omega} \leftarrow \hat{\omega}_0$

for $k = 1, \dots, T_w/\Delta t$ **do**

$\mathcal{S}.\text{step}()$

 ▷ warm-up before recording

end for

for $f = 0, \dots, F - 1$ **do**

for $k = 1, \dots, s$ **do**

$\mathcal{S}.\text{step}()$

end for

$\omega \leftarrow \mathcal{S}.\text{get_vorticity}()$

$D[b:b + B, f] \leftarrow \omega$

end for

end for

Remove trajectories containing non-finite values

$\text{save}(D)$

Quantity	Value
Physical system	2D Kolmogorov flow vorticity
Reynolds number	$Re = 1000$
Spatial resolution	256×256
Candidate trajectories	400
Recorded frames per trajectory	320
Stored dtype	<code>float32</code>
Initial condition	Gaussian random field
GRF parameters	$\alpha = 2.5, \tau = 7.0$
Forcing mode	$n_f = 4$
Time step	$\Delta t = 10^{-3}$
Linear drag	0.1
Numerical method	Pseudo-spectral, dealiased, Crank–Nicolson–Heun
Valid trajectories retained	314
Rejected trajectories	86

Table 4.1: Main construction details for the generated Kolmogorov-flow dataset.

4.2.3 Preprocessing and Splits

The generated trajectories were split by trajectory index into training, validation and test sets using an 80/10/10 split. The downsampling operator D used to construct low-resolution frames is implemented by stride subsampling by a factor of four in each spatial direction,

$$x_k^{LR} = D(x_k^{HR}) = x_k^{HR}[:, :4, :4].$$

Thus, a 256×256 high-resolution vorticity frame becomes a 64×64 low-resolution vorticity frame. The AR and SR modules read the same trajectories in different modes. The AR module uses a pair-based loader, where each sample is an input frame together with one or more future target frames. The SR module uses a frame-based loader, where each sample is a single high-resolution frame and its low-resolution condition is constructed by downsampling.

After downsampling, all fields were normalized by the standard deviation computed from the training split. The AR and SR modules therefore each used their own training-set normalization value. The same normalization value was then used for validation, testing, and joint inference, before predictions were scaled back to the original vorticity scale for visualization and final evaluation.

For the joint module, no separate training split is introduced. The joint pipeline uses the trained AR and SR modules and is evaluated on held-out test trajectories by first producing a low-resolution AR rollout and then applying the SR module to the predicted low-resolution frames.

Module	Dataset mode	Train	Validation	Test
AR	Consecutive low-resolution pairs	252 trajectories	31 trajectories	31 trajectories
SR	Individual high-resolution frames	254 trajectories	30 trajectories	30 trajectories

Table 4.2: Trajectory-level train, validation and test splits used for the AR and SR modules. Both splits follow an approximately 80/10/10 split of the 314 valid generated trajectories.

For the AR module, one-step training uses consecutive frame pairs. Therefore, a trajectory of length 320 gives 319 AR samples, since the final frame has no following frame to form an input-target pair:

$$(x_0, x_1), (x_1, x_2), \dots, (x_{318}, x_{319}).$$

The final frame x_{319} is still used as the target in the last pair, but it is not used as an input frame for one-step AR training. In contrast, the SR module treats frames independently. Therefore, a trajectory of length 320 gives 320 SR samples, since every frame can be downsampled to construct its low-resolution condition and used as its own high-resolution target:

$$x_0, x_1, \dots, x_{319}.$$

The dataset is therefore not only a storage detail, but part of the experimental setup. If the model is trained on a small or transient dataset, then it becomes difficult to separate whether a poor rollout is caused by the model or by the data. For this reason, the later experiments should be read together with the dataset choice that produced them. After the dataset has been defined, the next section introduces the AR module and its training procedure, followed by the SR module.

4.3 AR Module

The autoregressive module is a Koopman autoencoder acting on low-resolution vorticity fields. The encoder maps a low-resolution frame to a latent state, the latent state is propagated forward in time by a learned linear operator, namely a finite matrix, and the decoder maps the next latent state back to pixel space. If E_θ denotes the encoder, G_θ denotes the decoder and K denotes the forward latent operator, then one-step forward prediction is written as

$$\hat{x}_{k+1}^{LR} = G_\theta (K E_\theta(x_k^{LR})) .$$

Consequently, this also allows a low-resolution vorticity field to be pushed forward by n saved time steps. However, although this property is important for rollout, the scope of this thesis is primarily based on learning one-step transitions,

$$\hat{x}_{k+n}^{LR} = G_\theta (K^n E_\theta(x_k^{LR})) .$$

If K is orthogonal, then its inverse would also allow backward evolution in time by applying the transpose transformation in latent space using $K^\top$. The implementation of the backward operator allows two ways of handling backward dynamics. The first is the single-operator variant, where only the forward operator K is independently learned. When a backward prediction is needed, the backward map is therefore taken to be $K^\top$. Thus, the same learned operator supplies both the forward map and a tied backward map. This does not make $K^\top$ an exact inverse in general, but if K is close to orthogonal, then $K^\top$ can act as an approximate inverse.

In the separate-operator variant, the model uses a distinct backward operator B . This operator is initialised from the forward operator but is then learned as a separate parameter. In this case, the backward decoded prediction is written without a transpose as

$$\hat{x}_k^{LR} = G_\theta (B E_\theta(x_{k+1}^{LR})) .$$

The backward operator is not used to produce the final forward rollout. It is used during training as an additional reverse-time constraint on the learned latent dynamics.

4.3.1 Training Objective

The AR loss is constructed from a small number of reusable terms. The forward loss compares the predicted next frame to the true next frame in pixel space. The reconstruction loss keeps the encoder and decoder tied to the input frame, also in pixel space. The backward loss, when used, asks the backward operator to reconstruct the current state from the next state. Additional latent, trace-consistency, and orthogonality terms were used in selected experiments to align predicted and encoded latent states, encourage compatibility between forward and backward operators, and control the spectrum of K during long-horizon rollouts from the latent space.

Before defining each loss term in the following paragraphs, let a training example be written as $(x, y_1, \dots, y_R)$, where $x = x_k^{LR}$ and $y_j = x_{k+j}^{LR}$ for $j = 1, \dots, R$. Here, R denotes the number of pixel-space rollout steps used during training. The symbol $\ell(\cdot, \cdot)$ denotes the pixel-space discrepancy used inside the forward, backward, and reconstruction losses. In this thesis, ℓ is chosen either

as mean squared error or as relative L^2 error, depending on the experiment. The exact choice is reported in the corresponding experiment.

For two fields a and b , the mean squared error is

$$\ell_{\text{MSE}}(a, b) = \frac{1}{HW} \sum_{p=1}^H \sum_{q=1}^W (a_{p,q} - b_{p,q})^2,$$

while the relative L^2 error is

$$\ell_{\text{rel}, L^2}(a, b) = \frac{\|a - b\|_2}{\|b\|_2}.$$

Given that, a general form of the loss is

$$\begin{aligned} \mathcal{L} = & \lambda_{\text{fwd}} \mathcal{L}_{\text{fwd}} + \lambda_{\text{recon}} \mathcal{L}_{\text{recon}} + \lambda_{\text{bwd}} \mathcal{L}_{\text{bwd}} \\ & + \lambda_{\text{latent, fwd}} \mathcal{L}_{\text{latent, fwd}} + \lambda_{\text{consist}} \mathcal{L}_{\text{consist}} \\ & + \lambda_{\text{ortho}, K} \mathcal{L}_{\text{ortho}, K} + \lambda_{\text{ortho}, B} \mathcal{L}_{\text{ortho}, B} \end{aligned}$$

Not every experiment uses every term, and some experiments disable selected loss terms. The active loss terms are stated in the corresponding experiments section. The following paragraphs define these loss terms and their intended roles.

Pixel space forward loss The forward loss compares decoded future predictions against the true future frames. Starting from $z_0 = E_\theta(x)$, the predicted frame at rollout step j is

$$\hat{y}_j = G_\theta(K^j z_0).$$

The pixel-space forward loss is then

$$\mathcal{L}_{\text{fwd}} = \frac{1}{R} \sum_{j=1}^R \ell(\hat{y}_j, y_j).$$

This is the fundamental prediction term. If only this term is used, the model is mainly asked to make the decoded next frames look correct, independently of spectral constraints on K . In the experiments of this thesis, R is set to one.

Pixel space backward loss The backward loss applies the backward operator from the final target frame in the rollout. Let $z_R = E_\theta(y_R)$. The backward prediction after j steps is

$$\hat{b}_j = G_\theta(B^j z_R).$$

Its target is the previous frame in the sequence, namely y_{R-j} when $j < R$, and x when $j = R$. Therefore,

$$\mathcal{L}_{\text{bwd}} = \frac{1}{R} \sum_{j=1}^R \ell(\hat{b}_j, \tilde{y}_j), \quad \tilde{y}_j = \begin{cases} y_{R-j}, & j < R, \\ x, & j = R. \end{cases}$$

This term does not define the final forward rollout. It is used as an extra training constraint so that the learned latent dynamics are not only pushed in one temporal direction. As with the

pixel-space forward loss, $R = 1$ has been the main case throughout this thesis.

Pixel space reconstruction loss The reconstruction loss decodes the latent representation without applying the time-evolution operator:

$$\hat{x} = G_\theta(E_\theta(x)).$$

The loss is

$$\mathcal{L}_{\text{recon}} = \ell(\hat{x}, x).$$

This term keeps the combined encoder E_θ and decoder G_θ close to an identity mapping on the input frame. It prevents the latent space from becoming useful only for dynamics while becoming poor at representing the frame being encoded.

Latent space forward loss The latent forward loss compares the evolved latent state directly against the encoded future frame. If L latent rollout steps are active, then

$$\mathcal{L}_{\text{latent, fwd}} = \frac{1}{L} \sum_{j=1}^L \|K^j E_\theta(x) - E_\theta(y_j)\|_2^2.$$

Unlike the pixel forward loss, this term does not wait until the decoder. It asks the learned linear operator to land near the latent representation that the encoder would have assigned to the true future frame.

Operator trace loss The trace consistency loss uses the forward and backward operator matrices directly. In the implementation this term is

$$\mathcal{L}_{\text{consist}} = \left| \frac{\text{tr}(KB)}{\text{tr}(K) \text{tr}(B)} - 1 \right|.$$

This is a lightweight operator-level constraint. It does not compare predicted frames directly. Instead, it encourages the learned forward and backward operators to remain compatible through a trace-based summary.

In the single-operator case, $B = K^\top$, so the same definition applies after this substitution.

Operator orthogonality loss The orthogonality loss is applied separately to the forward and backward operators. For the forward operator,

$$\mathcal{L}_{\text{ortho}, K} = \|K^\top K - I\|_F^2,$$

and for the backward operator,

$$\mathcal{L}_{\text{ortho}, B} = \|B^\top B - I\|_F^2.$$

This term is important because repeated rollout can amplify small spectral errors. If the operator is closer to orthogonal, then its transpose is closer to an inverse and latent norms are less likely to grow or collapse immediately. This is especially relevant here because this thesis mainly focuses

on learning single-step dynamics for practical reasons. The long-horizon rollout depends not only on one-step prediction accuracy, but also on the spectral behaviour of the learned latent operator. Therefore, the orthogonality penalty acts as an additional constraint on repeated applications of K or B , although it does not guarantee physically correct rollout behaviour.

In the single-operator case, only the forward matrix is penalized once.

Architecture Summary Before defining the training procedure, it is useful to describe the structure of the AR module. The architecture is based on a Restormer-style transformer backbone [Zamir et al., 2022], adapted here for low-resolution vorticity prediction. The main path consists of an encoder, a latent linear dynamics module, and a decoder.

The AR module takes a low-resolution vorticity field as input. In this thesis, this field is represented as a single-channel 64×64 image. Before the field enters the encoder, two coordinate channels are concatenated to it. Thus, the model receives both the vorticity value and the spatial position associated with each grid point.

The encoder starts with a 3×3 convolution that produces an overlapping patch embedding. It then applies groups of transformer blocks separated by downsampling layers. The downsampling schedule is $(2, 4, 4)$, meaning that the spatial resolution is progressively reduced by these factors. Therefore, an input field of size 64×64 is reduced to a latent grid of size 2×2 .

Each transformer block contains two main components. The first component is a transposed attention layer, which computes attention across feature channels. The second component is a gated depthwise-convolution feed-forward network. Thus, the encoder should not be read as a plain convolutional network, but as a transformer-style image encoder with convolutional operations inside each block.

After encoding, the 2×2 latent grid is flattened into a single latent vector. The Koopman operator K acts on this vector as a finite-dimensional linear map. In the main AR configuration, this latent vector has dimension 512, so the full-rank forward operator is represented by a 512×512 matrix.

The decoder reverses the encoder structure. It reshapes the evolved latent vector back into a 2×2 grid, applies upsampling layers and transformer blocks, and finally uses a 3×3 convolution to return to a single-channel vorticity field. Therefore, the full AR module can be read as an encode–evolve–decode model, where the low-resolution field is encoded, the latent state is evolved linearly, and the evolved state is decoded back to pixel space. A later experiment also considers an alternative inference method, which weakens the strict Koopman interpretation by reintroducing the nonlinear encoder-decoder pair at every rollout step.

Configuration	Koopman small	Koopman big
Base width	4	16
Transformer blocks	(2, 2, 2, 2)	(4, 6, 6, 8)
Attention heads	(1, 2, 4, 4)	(1, 2, 4, 8)
Downsampling schedule	(2, 4, 4)	(2, 4, 4)
Latent grid size	2×2	2×2
Latent vector dimension	128	512
Forward operator size	128×128	512×512
Total parameters	180,912	4,534,872

Table 4.3: AR architecture configurations used in the experiments.

The small configuration was used in early experiments and is specified where relevant in the experiments section, while the big configuration became the main AR configuration used later. The main difference is the channel width and the number of transformer blocks, which indirectly changes the size of the learned Koopman operator. In addition, the total parameter counts were calculated for the separate-operator case.

Training procedure The next natural step is to understand how these loss terms combine during training of the AR module. The procedure can be summarised as follows.

Algorithm 2 Koopman autoencoder training

Require: Dataset D , batch size B_s , epochs E , learning rate η , weight decay γ

Require: active loss weights λ_{fwd} , λ_{recon} , λ_{bwd} , $\lambda_{\text{latent,fwd}}$, λ_{consist} , $\lambda_{\text{ortho},K}$, $\lambda_{\text{ortho},B}$

Ensure: Trained encoder E_θ , decoder G_θ , forward operator K and optional backward operator B

Split D by trajectory index into train, validation and test sets

Downsample frames from 256×256 to 64×64

Build loader over rollout tuples $(x, y_1, \dots, y_R)$

Initialise Koopman AE (E_θ, K, G_θ) and, if used, backward operator B

Initialise optimiser **Adam** (θ, η, γ)

for $e = 1, \dots, E$ **do**

for $(x, y_1, \dots, y_R)$ in train loader **do**

$z_0 \leftarrow E_\theta(x)$

$\hat{y}_j \leftarrow G_\theta(K^j z_0)$ for $j = 1, \dots, R$

$\hat{x} \leftarrow G_\theta(z_0)$

 Compute $\mathcal{L}_{\text{fwd}}$ and $\mathcal{L}_{\text{recon}}$

if backward dynamics are active **then**

 Compute $\mathcal{L}_{\text{bwd}}$ from $E_\theta(y_R)$ using B or $K^\top$

end if

if latent rollout loss is active **then**

 Compute $\mathcal{L}_{\text{latent,fwd}}$

end if

if trace consistency is active **then**

 Compute $\mathcal{L}_{\text{consist}}$ using B or $K^\top$

end if

if orthogonality regularisation is active **then**

```

    Compute  $\mathcal{L}_{\text{ortho},K}$  and, if present,  $\mathcal{L}_{\text{ortho},B}$ 
  end if
  Form the weighted loss  $\mathcal{L}$  from the active terms
  Update  $E_\theta$ ,  $G_\theta$ ,  $K$  and, if present,  $B$  using backpropagation
end for
end for

```

The AR training procedure also influences the inference mode used in this thesis, leading to two alternative methods of inference.

4.3.2 Inference Modes

Two rollout modes are important in this thesis. The first is strict Koopman inference, where the initial frame is encoded once and the same latent operator is iterated in latent space,

$$z_{k+n} = K^n E_\theta(x_k^{LR}), \quad \hat{x}_{k+n}^{LR} = G_\theta(z_{k+n}).$$

The second is sequential inference. In this case, each predicted frame is decoded and then used as the next input,

$$\hat{x}_{k+1}^{LR} = G_\theta(K E_\theta(\hat{x}_k^{LR})).$$

The distinction matters because strict Koopman inference tests whether one latent linear trajectory remains meaningful over many steps, while sequential inference repeatedly returns through the learned pixel-space representation. This will be discussed later in the discussion section, where sequential inference is interpreted as a more practical rollout mode that works at the cost of reintroducing nonlinear encoder-decoder dynamics into the time-stepping process. Thus, the inference mode is not set globally across experiments and is specified where relevant in the experiments section.

After defining the inference modes used for long-horizon rollouts, the next step is to define how the resulting predictions are evaluated.

4.3.3 Evaluation

The validity and physical accuracy of such low-resolution rollouts are partly assessed by visual inspection over the validation rollouts, but this is not sufficient on its own. For that reason, the AR module is evaluated using several complementary diagnostics during the same validation run, namely rollout visualizations, rollout MSE, temporal autocorrelation, energy spectra, Q-Q plots, and Koopman-operator eigenvalue statistics.

Rollout visualizations Given an initial frame x_0^{LR} , the AR model produces a predicted rollout

$$\hat{x}_1^{LR}, \hat{x}_2^{LR}, \dots, \hat{x}_T^{LR}.$$

This rollout is compared against the ground-truth sequence

$$x_1^{LR}, x_2^{LR}, \dots, x_T^{LR}.$$

For selected frame indices k , the visualization shows three fields, namely the ground truth x_k^{LR} , the prediction $\hat{x}_k^{LR}$, and the absolute error

$$e_k = |\hat{x}_k^{LR} - x_k^{LR}|.$$

This visual comparison should be interpreted carefully because, for chaotic flows, small differences in the initial or predicted state can grow over a Lyapunov time scale, so long-horizon frame-by-frame similarity is not expected. However, it is useful for identifying qualitative failure modes in the rollout, such as blow-up, collapse, over-smoothing, or persistent spatial artifacts.

Rollout mse The rollout MSE gives a pointwise error curve over the rollout horizon. For a set of M trajectories, the error at prediction step k is computed as

$$\text{MSE}(k) = \frac{1}{M} \sum_{m=1}^M \frac{1}{HW} \sum_{p=1}^H \sum_{q=1}^W \left(\hat{x}_k^{LR,(m)}(p, q) - x_k^{LR,(m)}(p, q) \right)^2.$$

This measures how quickly the predicted fields move away from the corresponding ground-truth fields in pixel space. Similar to rollout visualizations, this metric should be interpreted carefully for long horizons because chaotic trajectories can separate over long rollouts, even when their statistics remain physically reasonable.

Temporal autocorrelation Temporal autocorrelation measures how similar a trajectory is to itself after a time lag. In this thesis, $R(h)$ denotes the autocorrelation at lag h . The value $R(0)$ is the zero-lag autocorrelation, where each frame is compared to itself, and is used to normalize the curve so that $R(0) = 1$.

For a trajectory of vorticity fields $x_0, \dots, x_T$, each frame is first flattened into a vector. The unnormalized autocorrelation at lag h is computed as

$$C(h) = \frac{1}{T-h+1} \sum_{k=0}^{T-h} \langle x_k, x_{k+h} \rangle.$$

The normalized autocorrelation is then

$$R(h) = \frac{C(h)}{C(0)}.$$

For plotting, this integer lag is also reported as the normalized lag

$$k^* = \frac{h}{T},$$

so the corresponding plotted curve is written as $R(k^*)$.

For example, $R(1)$ compares neighbouring frames, while $R(10)$ compares frames ten time steps apart. The same calculation is applied to the ground-truth trajectory and to the predicted rollout. This metric is useful because it checks whether the prediction preserves the temporal structure of the flow, even when exact frame-by-frame similarity becomes impossible over long horizons.

For the later capacity and loss-weight sweeps, the autocorrelation curve is also reduced to a

scalar score. Let $\bar{R}_{GT}(h)$ and $\bar{R}_{model}(h)$ denote the validation-set mean autocorrelation curves. The score is

$$S_R = \text{clip}_{[0,1]} \left(1 - \frac{\int |\bar{R}_{model}(h) - \bar{R}_{GT}(h)| dh}{\int |\bar{R}_{GT}(h)| dh + 10^{-8}} \right).$$

In the implementation, the integrals are computed by trapezoidal integration over the sampled lag grid. This score is not used as the only evidence for a rollout, but as a compact way to rank sweep configurations before checking rollout frames, spectra, and Q-Q diagnostics.

Energy spectrum The energy spectrum measures how the spatial content of a vorticity field is distributed across wavenumbers. For each frame x_k , the two-dimensional Fourier transform $\hat{x}_k(\kappa_x, \kappa_y)$ is computed, and the power at each Fourier mode is defined as

$$P_k(\kappa_x, \kappa_y) = \frac{|\hat{x}_k(\kappa_x, \kappa_y)|^2}{H^2 W^2}.$$

The radial wavenumber is then defined by

$$\kappa = \text{round} \left(\sqrt{\kappa_x^2 + \kappa_y^2} \right).$$

The spectrum $E(\kappa)$ is obtained by averaging $P_k(\kappa_x, \kappa_y)$ over all modes whose radial wavenumber falls in the same bin. This is computed for both the ground-truth trajectories and the predicted rollouts and compared together. If the predicted $E(\kappa)$ matches the ground-truth $E(\kappa)$, then the model has similar spatial scale statistics. This does not mean that the predicted vortices are in the correct location. It only means that the prediction has the right amount of structure compared to the ground-truth trajectory.

Q-Q plots Q-Q plots compare the distribution of vorticity values in the predicted rollout against the distribution of vorticity values in the ground-truth rollout. All vorticity values from the selected frames are flattened into one set of predicted values and one set of ground-truth values. For a quantile level $q \in [0, 1]$, let $Q_{\text{pred}}(q)$ denote the predicted quantile and let $Q_{\text{GT}}(q)$ denote the ground-truth quantile. The Q-Q plot shows the points

$$(Q_{\text{pred}}(q), Q_{\text{GT}}(q))$$

for many values of q .

For example, low quantiles compare the lower-vorticity tail, the median quantile compares the central value, and high quantiles compare the high-vorticity tail. If the predicted and ground-truth distributions match, the points lie close to the diagonal line. This metric is useful because it checks whether the rollout has the correct range and tail behaviour, even when exact spatial alignment is imperfect. In the implementation, the number of quantiles is set to 1000, giving 1000 evenly spaced quantile comparisons.

Operator diagnostics For Koopman-based rollouts, the learned linear operators are also monitored directly. Independent of validation set plotting of previously mentioned metrics, operator diagnostics are monitored directly per epoch. Such diagnostics include the eigenvalues of the

forward learned operator K and the backward operator B . The minimum, mean and maximum magnitudes of these eigenvalues are then recorded,

$$\min_i |\lambda_i(K)|, \quad \frac{1}{d} \sum_{i=1}^d |\lambda_i(K)|, \quad \max_i |\lambda_i(K)|,$$

with the same quantities computed for B . These diagnostics are useful because long-horizon rollout repeatedly applies the same operator in the Koopman inference case. If eigenvalue magnitudes move far above one, errors may grow. Likewise, if they move far below one, latent states may collapse and the decoder may struggle to extract meaningful information under repeated rollout. This interpretation is returned to in the discussion section.

4.3.4 Latent Least-Squares Probe

This experiment is designed to separate two different problems concerning the AR module and its capability. The purpose of this split is first to ask whether the autoencoder latent space admits an approximately linear time step, and then to infer whether gradient-based training can learn such a time step. To separate the two questions, the autoencoder is trained using reconstruction only, giving frozen weights E_{θ^*} and G_{θ^*} . The latent operator is then fitted directly by least squares on the space of latents encoded from the frozen weights.

The procedure starts with training the autoencoder only, without the learned dynamics or the K operator. Then, using E_{θ^*} , each pair of low-resolution vorticity fields (x_k^{LR}, x_{k+1}^{LR}) is encoded into latent pairs (z_k, z_{k+1}) . Given these latent pairs, K^* is fitted for the forward dynamics case only,

$$x_k \xrightarrow{E_{\theta^*}} z_k, \quad x_{k+1} \xrightarrow{E_{\theta^*}} z_{k+1},$$

$$K^* = \arg \min_K \sum_k \|z_{k+1} - K z_k\|_2^2.$$

This probe tests whether the latent space itself supports a linear time step, independent of whether gradient descent can learn the operator. Therefore, the LSQ probe should be read as a best-case test for linear latent evolution. It does not prove that this is the best possible latent space for Koopman prediction, since joint training of the autoencoder and operator may produce a different representation. This assumption is discussed later.

The AR module is therefore not only being tested as a one-step predictor. The main question is whether the learned latent representation can survive repeated time evolution. This is why this thesis keeps returning to rollout plots, spectra, and eigenvalues instead of relying only on one-step training loss.

4.4 SR Module

The super-resolution module maps a low-resolution vorticity field x_k^{LR} to a high-resolution vorticity field x_k^{HR} . It is implemented as a conditional diffusion model in the style of SR3 [Saharia et al., 2023]. During training, the high-resolution target field is corrupted with Gaussian noise, while a low-resolution condition is constructed by downsampling the high-resolution field and then upsampling it back to the high-resolution grid using an upsampling operator U . This mimics the

coarse conditioning signal that the SR module receives from the AR module during joint inference. The upsampled condition is written as

$$c_k = U(x_k^{LR}).$$

The denoiser receives a noisy high-resolution field $x_{k,\tau}$, the diffusion step τ , and the condition c_k , and predicts the noise contained in $x_{k,\tau}$,

$$\hat{\epsilon} = \epsilon_\phi(x_{k,\tau}, \tau, c_k).$$

4.4.1 Training Objective

The SR module is trained with a single denoising objective [Ho et al., 2020]. For each high-resolution training frame x_k^{HR} , a diffusion step τ is sampled and Gaussian noise ϵ is added to form a noisy field $x_{k,\tau}$. The denoiser ϵ_ϕ is then given the noisy field, the diffusion step, and the low-resolution condition c_k , and is trained to predict the injected noise,

$$\mathcal{L}_{\text{SR}} = \|\epsilon_\phi(x_{k,\tau}, \tau, c_k) - \epsilon\|_2^2.$$

The SR module therefore does not learn the high-resolution field directly in one forward pass. It learns how to remove noise at different diffusion levels while being guided by the low-resolution condition. During inference, this denoising step is applied repeatedly inside the reverse diffusion process to generate a high-resolution reconstruction from the condition c_k . No perceptual, adversarial, autoregressive, or physics-based loss terms were used for the SR module.

Architecture summary Before defining the full training procedure, it is useful to define the structure of the SR module. In short, the SR module is based on a U-Net architecture adapted for the SR3-style conditional denoising task.

The SR module accepts 256×256 single-channel fields, matching the dimensionality of the high-resolution vorticity fields used in this thesis. The noisy high-resolution field x_τ is first embedded through a convolutional pathway, while the upsampled low-resolution condition c_t is embedded through a separate conditioning pathway. These two representations are then concatenated and passed into the main U-Net body.

The implemented U-Net follows a standard downsampling and upsampling structure, with attention applied at selected spatial resolutions. The network uses residual blocks inside the downsampling and upsampling paths, while U-Net skip connections carry features from the downsampling path to the corresponding upsampling path. The final output has the same spatial resolution as the input, but represents the predicted noise $\hat{\epsilon}$ rather than the reconstructed field directly.

In the main SR configuration, the denoiser uses one input channel and one output channel, matching the scalar vorticity field. The base latent width is 64, the channel multipliers are (1, 1, 1, 2), and one residual block is used at each resolution. Attention is applied at spatial resolution 16. The model operates on 256×256 fields and contains 3,511,745 trainable parameters.

Configuration	SR U-Net
Input channels	1
Output channels	1
Image resolution	256×256
Base latent width	64
Channel multipliers	(1, 1, 1, 2)
Residual blocks per resolution	1
Attention resolution	16
Condition downsampling factor	4
Total parameters	3,511,745

Table 4.4: SR architecture configuration used in the experiments.

The SR configuration is therefore fixed for the reported SR experiments.

Training Procedure The SR training procedure can be summarised as follows. Unlike the AR module, the SR module tracks two copies of the model weights during training. Here, ϕ denotes the trainable SR denoiser parameters, while ψ denotes the EMA copy used for sampling. The EMA, or Exponential Moving Average [Polyak and Juditsky, 1992], is used in this thesis to smooth the parameter trajectory of the diffusion model across training epochs.

Algorithm 3 Conditional diffusion super-resolution training

Require: Dataset D , batch size B_s , epochs E , learning rate η , EMA decay ρ

Require: upscale factor $r = 4$, diffusion schedule $\{\beta_\tau\}_{\tau=1}^{\tau_{\max}}$

Ensure: Trained denoising network ϵ_ϕ and EMA parameters ψ

Split D by trajectory index into train, validation and test sets

Build loader over individual high-resolution frames x^{HR}

Initialise denoising U-Net ϵ_ϕ and diffusion manager

Initialise EMA parameters $\psi \leftarrow \phi$

Initialise optimiser $\text{Adam}(\phi, \eta)$

for $e = 1, \dots, E$ **do**

for x^{HR} in train loader **do**

$x^{LR} \leftarrow D(x^{HR})$

$c \leftarrow \text{upsample}(x^{LR}, r)$

$\tau \sim \text{Uniform}\{1, \dots, \tau_{\max}\}$ and $\epsilon \sim \mathcal{N}(0, I)$

$x_\tau \leftarrow \sqrt{\bar{\alpha}_\tau} x^{HR} + \sqrt{1 - \bar{\alpha}_\tau} \epsilon$

$\hat{\epsilon} \leftarrow \epsilon_\phi(x_\tau, \tau, c)$

$\mathcal{L}_{\text{SR}} \leftarrow \|\hat{\epsilon} - \epsilon\|_2^2$

 Update ϕ using backpropagation

$\psi \leftarrow \rho\psi + (1 - \rho)\phi$

end for

end for

Validation and test denoising losses are reported using the live model ϵ_ϕ , while generated samples and joint-module SR outputs use the EMA model ϵ_ψ .

Inference Mode At inference time, the SR module starts from Gaussian noise and repeatedly updates the current sample while conditioning on c_k . At a diffusion step τ , the EMA denoiser predicts

$$\hat{\epsilon}_{k,\tau} = \epsilon_\psi(x_{k,\tau}, \tau, c_k).$$

This prediction is used to estimate the clean high-resolution field,

$$\hat{x}_{k,0} = \frac{x_{k,\tau} - \sqrt{1 - \bar{\alpha}_\tau} \hat{\epsilon}_{k,\tau}}{\sqrt{\bar{\alpha}_\tau}}.$$

The sample is then moved to the next selected diffusion step τ' using the DDIM-style update

$$x_{k,\tau'} = \sqrt{\bar{\alpha}_{\tau'}} \hat{x}_{k,0} + \sqrt{1 - \bar{\alpha}_{\tau'}} \hat{\epsilon}_{k,\tau}.$$

Rather than using the full DDPM reverse chain, this thesis uses a DDIM-style sampler [Song et al., 2021] with fewer selected diffusion steps. In the main configuration, the training schedule has $\tau_{\max} = 1000$ diffusion steps, while sampling uses 40 inference steps.

4.4.2 Evaluation

The main numerical evaluation for the SR module is the diffusion denoising loss. On validation and test frames, the model is given a noisy high-resolution field $x_{k,\tau}$, a diffusion step τ , and the upsampled low-resolution condition c_k . The reported loss is the mean squared error between the predicted noise $\epsilon_\phi(x_{k,\tau}, \tau, c_k)$ and the injected noise ϵ .

Unlike the AR module, the SR module is not evaluated as a time-evolution model. Thus, temporal diagnostics such as autocorrelation are not used in the standalone SR evaluation. Instead, the SR module is evaluated as a conditional reconstruction model using qualitative sample visualizations, absolute-error maps, and energy-spectrum comparisons.

Additionally, in this thesis the reported validation and test denoising loss is computed using the trainable network ϵ_ϕ . The generated DDIM samples and joint-module SR outputs use the EMA version ϵ_ψ .

High-resolution visualizations The SR module is also evaluated through generated high-resolution samples. For a small number of validation or test frames, the visualization compares the ground-truth high-resolution frame x_k^{HR} , the upsampled low-resolution condition c_k , and the DDIM sample $\hat{x}_k^{HR}$. This makes it possible to inspect whether the generated frame follows the large-scale structure of the condition while recovering finer spatial detail.

Absolute-error visualizations For the same samples, an absolute-error field is plotted as

$$e_k = |\hat{x}_k^{HR} - x_k^{HR}|.$$

This plot shows where the generated high-resolution field differs from the reference field in pixel space. It should be read as a spatial diagnostic rather than as a full physical metric, since the SR module is judged mainly by whether it reconstructs plausible high-resolution structure from the low-resolution condition.

Energy spectrum The SR module is further evaluated using energy-spectrum comparisons. The spectrum is computed for the ground-truth high-resolution frames, the upsampled low-resolution conditions, and the generated samples. This checks whether the generated samples recover spatial frequency content that is missing from the upsampled condition, especially at higher wavenumbers.

4.5 Joint Module

The joint module combines the AR and SR components into a single inference pipeline. First, an initial high-resolution test frame is downsampled to the low-resolution grid. The AR module then produces a low-resolution rollout. Finally, each predicted low-resolution frame is passed to the SR module to produce a high-resolution rollout.

The joint inference procedure can be summarised as follows. The AR stage may be run either in sequential mode or in Koopman mode, matching the inference modes defined previously.

Algorithm 4 Joint AR–SR rollout

Require: Trained AR model (E_θ, K, G_θ) and trained SR model ϵ_ψ

Require: AR inference mode $m \in \{\text{sequential}, \text{Koopman}\}$

Require: Test trajectories, rollout length R_{joint} , SR batch size B_{SR} , DDIM inference steps N_{DDIM}

Ensure: High-resolution predicted rollouts

Select test trajectories and extract initial frames x_0^{HR}

Downsample initial frames to obtain x_0^{LR}

for each selected trajectory **do**

$R^{LR} \leftarrow []$

if $m = \text{sequential}$ **then**

$u \leftarrow x_0^{LR}$

for $n = 1, \dots, R_{\text{joint}}$ **do**

$z \leftarrow E_\theta(u)$

$u \leftarrow G_\theta(Kz)$

Append u to R^{LR}

end for

else if $m = \text{Koopman}$ **then**

$z \leftarrow E_\theta(x_0^{LR})$

for $n = 1, \dots, R_{\text{joint}}$ **do**

$z \leftarrow Kz$

Append $G_\theta(z)$ to R^{LR}

end for

end if

Store R^{LR} as the predicted low-resolution rollout

end for

Concatenate predicted low-resolution frames from all trajectories

Upsample each low-resolution prediction to form SR condition c on the high-resolution grid

for each mini-batch of SR conditions **do**

$x \sim \mathcal{N}(0, I)$

for τ in the DDIM reverse schedule with N_{DDIM} steps **do**

$\hat{e} \leftarrow \epsilon_\psi(x, \tau, c)$

```

    Update  $x$  using the deterministic DDIM step
  end for
  Append the generated high-resolution frames
end for
Reshape generated frames into per-trajectory high-resolution rollouts

```

For the joint evaluation, the last 30 held-out test trajectories are used. The 32-frame and 320-frame stages are evaluated over all 30 trajectories. The 1024-frame stage is evaluated on one fixed held-out trajectory, chosen as the last trajectory in the held-out set, to inspect long-horizon qualitative behaviour and runtime.

The SR module should therefore be read differently from the AR module. The AR module is asked to move the system forward in time, while the SR module is asked to recover fine-scale structure from a low-resolution state. If the low-resolution state is already incorrect, the SR module can make it sharper, but it cannot make the AR trajectory correct again.

4.6 Implementation

All the modules and evaluation and experiments in this thesis were implemented in PyTorch. Training and inference were configured through Hydra, with one configuration file per entry point (`run_sim.py`, `train_ar.py`, `train_sr.py`, `run_joint.py`), where they respectively: generate the dataset, train the AR module, train the SR module and run joint module. The training runs were tracked with Weights & Biases. Random seeds were fixed through the Hydra configuration files so that dataset generation, training, and evaluation runs are reproducible. Training and evaluation were run on available NVIDIA GPU hardware, including RTX 4090, A100, and H200 devices. The methods do not depend on a specific GPU type, although runtime varied across available hardware. The code-base is available at github.com/Efesasa0/x01.

Chapter 5

Experiments and Results

This chapter presents the main experiments of the thesis and the results obtained from each stage. It begins with the dataset choices, including why the initial dataset was replaced by a generated Kolmogorov-flow dataset. It then turns to the autoregressive (AR) module, where the central question is whether a learned Koopman-style latent operator can produce stable and meaningful rollouts, and why sequential inference became an important alternative. The chapter then reports the super-resolution (SR) module, which reconstructs high-resolution vorticity structure from low-resolution conditions, before combining both modules in the final joint AR–SR pipeline. Some interpretation is included here because many design choices were discovered experimentally, and because the failures themselves are part of the narrative, while the broader implications are returned to in the discussion chapter.

5.1 Dataset Experiments

The first dataset used for early AR experiments was the 40-trajectory high-resolution Kolmogorov-flow dataset associated with [Shu et al., 2023]. Each trajectory contains 320 single-channel vorticity frames at spatial resolution 256×256 . This dataset was useful for testing the first AR code paths, but it was not ideal for the final experiments. The reason was that the trajectories contain early transient behaviour, whereas the main aim of the AR module is to study rollout behaviour in the developed chaotic regime. Supposing a cutoff point where the first 100 frames are treated as transient and removed, the effective amount of data becomes only 40×220 frames, with very few independent trajectories left for validation and test sets.

For this reason, a new Kolmogorov-flow dataset was constructed using the pseudo-spectral solver described in the method chapter. The target was to produce 400 candidate trajectories at $Re = 1000$, each with 320 recorded vorticity frames at resolution 256×256 . Initial conditions were sampled from a Gaussian random field with parameters $\alpha = 2.5$ and $\tau = 7.0$, and the solver used Kolmogorov forcing with mode $n_f = 4$, linear drag, dealiasing, and Crank–Nicolson–Heun time stepping.

As an example, Figure 5.1 shows two trajectories from both datasets. Visually, the generated dataset begins closer to the developed chaotic regime.

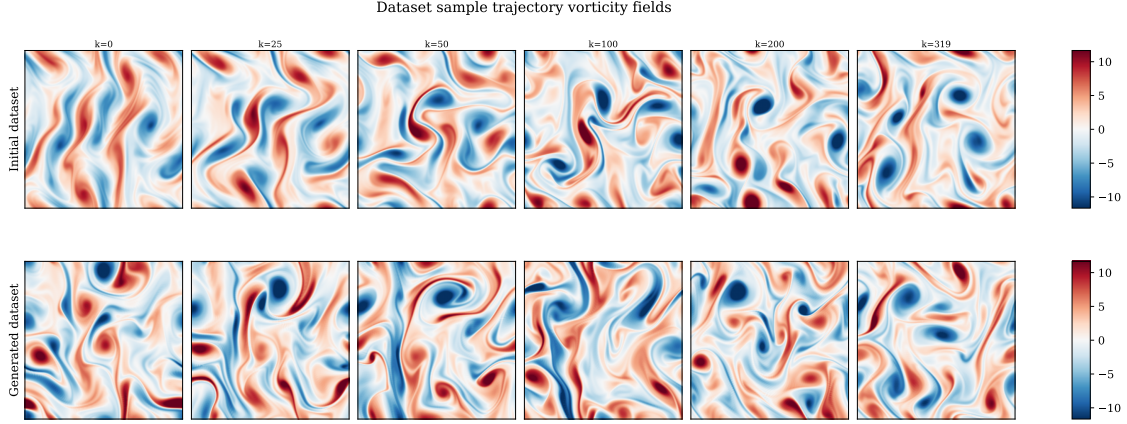


Figure 5.1: Example trajectories from the initial and generated datasets. The top row shows snapshots from one trajectory in the initial dataset, while the bottom row shows snapshots from one trajectory in the newly generated dataset.

The generated dataset was then cleaned before training. Any trajectory containing a non-finite value in any recorded frame was removed completely. This left 314 valid trajectories from the original 400, with 86 trajectories rejected. This filtering step is important because the learning problem should not be mixed with numerical failure cases. The aim is to study whether the AR and SR modules can learn from valid turbulent vorticity data, not whether they can repair invalid solver outputs.

The next step is to compare the initial and generated datasets in full, rather than only through example snapshots. Figure 5.2 shows four diagnostics. Panel (a) compares trajectory-to-trajectory spread by measuring the MSE of several trajectories against a reference trajectory, with the aim of showing whether different trajectories separate from one another instead of remaining nearly identical. Panel (b) shows where removed generated trajectories first became non-finite. Panels (c) and (d) compare the mean temporal autocorrelation and mean energy spectrum of the two datasets, respectively. These diagnostics are not intended as full physical validation of the simulator. Instead, they check whether the generated trajectories have reasonable vorticity-field statistics close to those of the initial dataset.

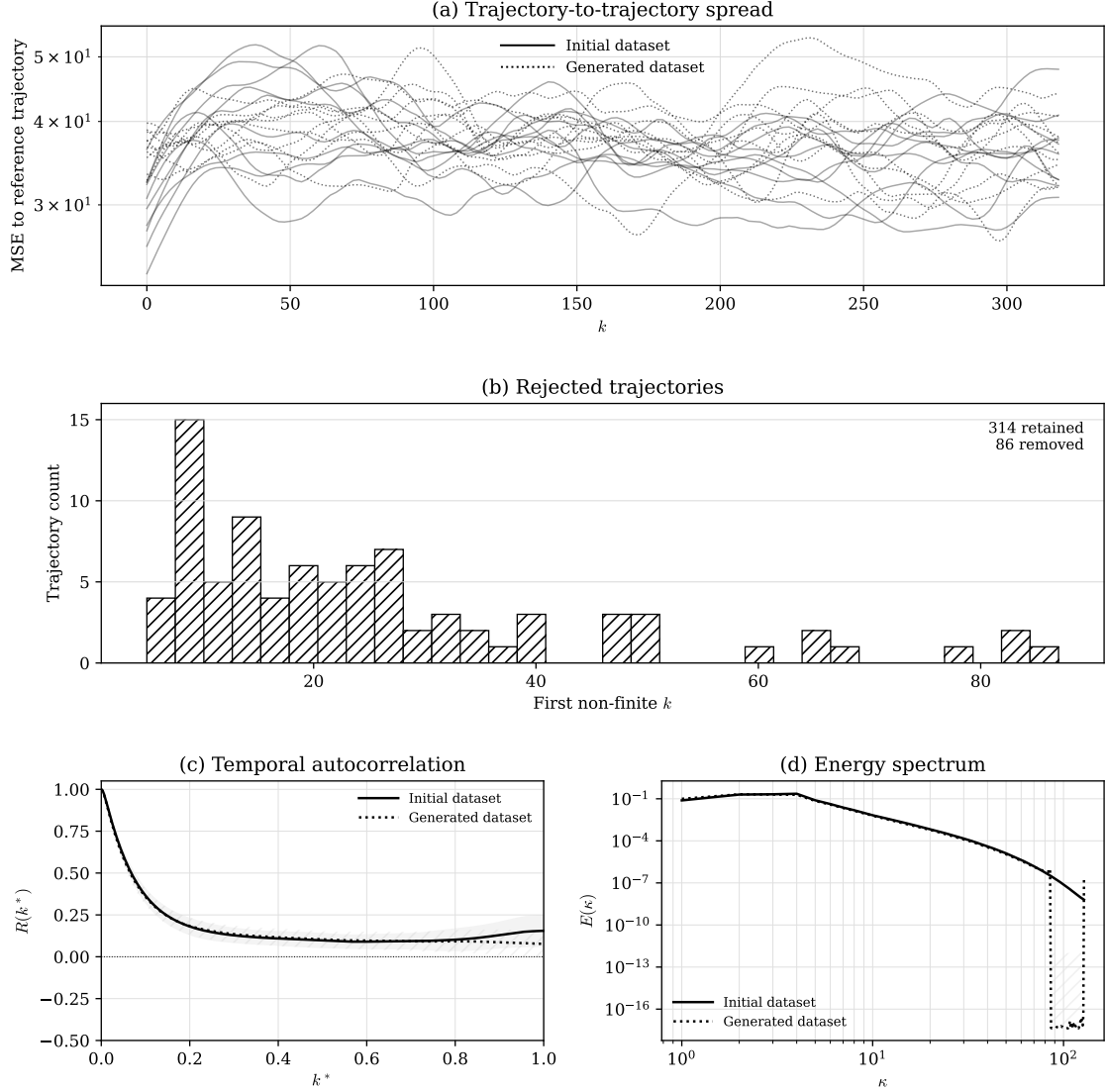


Figure 5.2: Dataset filtering and comparison diagnostics. (a) Comparison of 10 representative trajectories from each dataset used to compare temporal spread. (b) Histogram of eliminated trajectories from the newly generated dataset. (c) Comparison of the mean autocorrelation plots of both datasets. (d) Comparisons of the mean energy spectra plots of both datasets.

The resulting clean dataset was split by trajectory index. The AR and SR modules use the same cleaned dataset, but they read it in different forms. The AR module uses consecutive low-resolution frame pairs (x_k^{LR}, x_{k+1}^{LR}) , so each trajectory of length 320 contributes 319 one-step training pairs. The SR module treats frames independently, so each trajectory contributes 320 high-resolution reconstruction samples. For this reason, the split is best understood at both the trajectory level and the sample level.

Module	Sample type	Train	Validation	Test	Train samples
AR	Consecutive LR pairs	252	31	31	$252 \times 319 = 80,388$
SR	Individual HR frames	254	30	30	$254 \times 320 = 81,280$

Table 5.1: Trajectory-level splits and corresponding training, validation, testing sample counts used after filtering the generated dataset.

Thus, the generated dataset provides a more suitable supervised-learning setup for the later AR and SR experiments. It keeps the thesis focused on developed chaotic vorticity dynamics, while giving the validation and test sets enough independent trajectories to make the later rollout diagnostics more meaningful.

5.2 Autoregressive Module Experiments

This section reports the experiments for the autoregressive (AR) module. The AR module is the main time-evolution component of the project, since it takes a low-resolution vorticity field and attempts to produce a rollout forward in time. The central question is not only whether the next frame can be predicted, but whether the learned latent dynamics remain meaningful when the same operator is applied repeatedly. For that reason, the section begins with exploratory experiments that exposed the main failure modes, then moves to more controlled probes and ablations, and finally reports the sequential-inference setting that became the most practically useful rollout mode in this thesis.

When the small and big Koopman configurations are mentioned below, they refer to the architecture settings summarized in Table 4.3.

5.2.1 Exploratory Experiments

The exploratory experiments were conducted throughout much of this thesis, but only the most significant milestones are reported here. Many smaller ablations were useful during development, but were not interpretable enough to be presented as controlled experiments. Therefore, this subsection should be read as a record of observed failure modes rather than as a set of clean ablations where only one variable changes at a time. It is also intended to provide intuition for the reader.

Numerical instability The first major failure mode observed in the AR module was numerical instability under repeated Koopman rollout. This appeared in the earliest experiments, which used the small Koopman configuration and the initial 40-trajectory dataset with a 32/4/4 train/validation/test split. The high-resolution frames were downsampled from 256×256 to 64×64 . The loss terms were basic, namely forward and backward one-step losses, separately learned operators K and B , reconstruction loss, and trace-based consistency loss. At this stage, pixel-space losses were computed using MSE, and no direct orthogonality constraint was applied to K .

In this setting, the training loss decreased, but the long-horizon rollout was not stable and instead blew up numerically over repeated frames. The issue was likely linked to the spectrum of the learned forward operator. As the maximum eigenvalue magnitude of K moved above the

unit circle, repeated application of the same operator amplified latent components over the rollout horizon, leading to accumulated error in latent space and later to large pixel-space error after many applications of K .

Figure 5.3 gives the visual version of the same problem. The predicted rollout does not gradually become a slightly incorrect turbulent trajectory. Instead, the late frames become dominated by large-magnitude vorticity values.

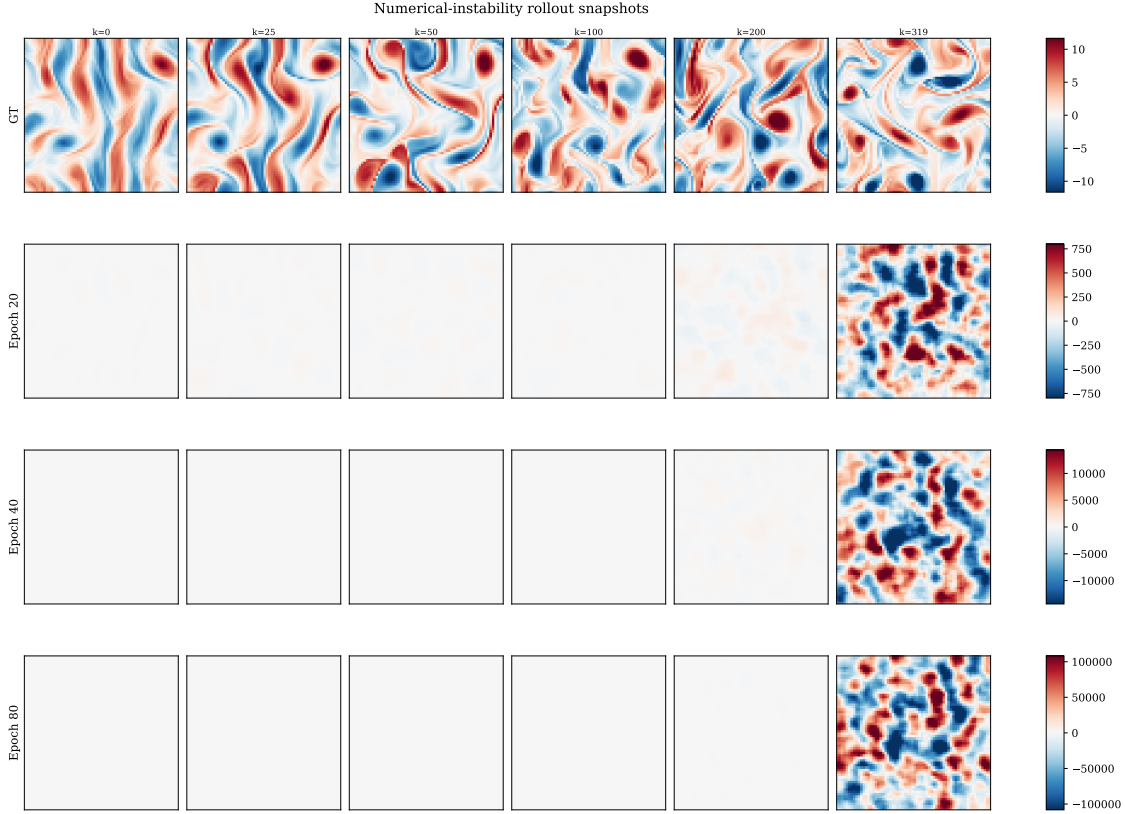


Figure 5.3: Validation rollouts for the numerical instability case. The rows compare the ground truth rollout with predictions from epochs 20, 40 and 80 at selected time steps.

Figure 5.4 summarizes this failure. Panel (a) shows the minimum, mean, and maximum eigenvalue magnitudes of the learned forward operator over training. Panel (b) shows the training and validation curves, where the training curve converges as validation error stops improving. Panel (c) shows the maximum rollout vorticity at saved epochs, showing that the blow-up appears early and becomes stronger with training.

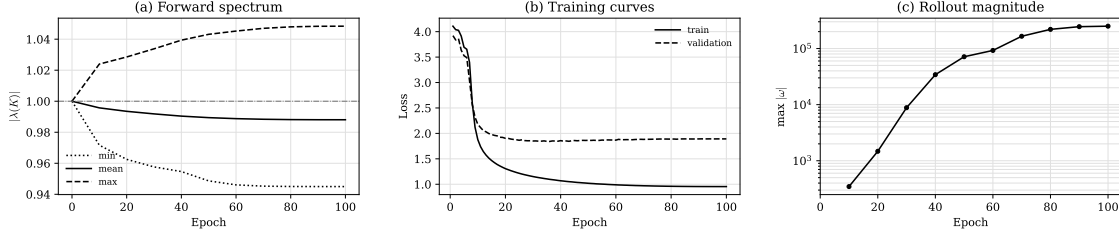


Figure 5.4: Diagnostics for the numerical instability case. (a) the evolution of the forward operator spectrum. (b) the training and validation losses. (c) the rollout vorticity magnitude across saved epochs.

Linear artifact propagation The second failure mode appeared after trying to avoid the previous blow-up by initializing the Koopman operator below the unit circle. Here, the same early setting was used, namely the small Koopman configuration, the initial 40-trajectory dataset, MSE-based pixel losses, reconstruction loss, and the trace-based consistency term. The additional change was initializing the spectrum of the forward operator K below the unit circle rather than near or above it, done by setting `init_scale=0.9`. This parameter simply multiplied the already initialized Koopman operator, which was approximately on the unit circle, by a scalar below one.

Figure 5.5 shows this bounded failure mode. Panel (a) shows that the forward spectrum stays below the unit circle, so the instability from the previous experiment is largely solved. Panel (b) shows training and validation losses that decrease and then flatten similarly to the blow-up case. Panel (c) shows that the rollout magnitude remains on the same order as the data, rather than increasing to unrealistic scales.

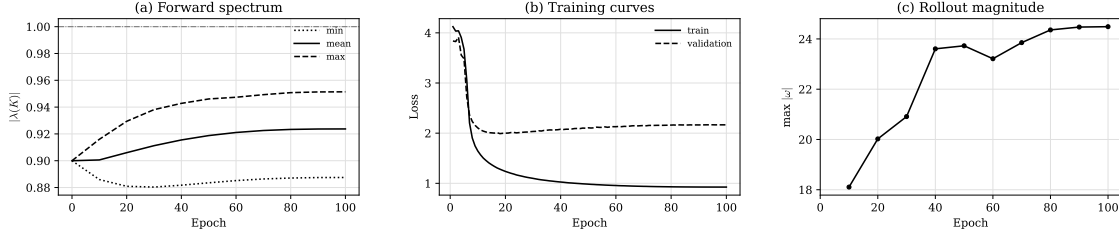


Figure 5.5: Diagnostics for the artifact-propagation case. (a) the forward operator spectrum of the learned forward K operator. (b) the training and validation losses from learning. (c) the magnitude of maximum rollout vorticities generated by AR across saved epochs.

However, Figure 5.6 shows why this was not a satisfactory solution. The predicted rollouts remain bounded, but later frames contain structured artifacts that are not present in the ground-truth trajectory. These artifacts are especially visible at later time steps, where the prediction develops a grid-like pattern that becomes a dead end for the rollout. This shows that spectral damping alone is too weak as a change for a good Koopman rollout.

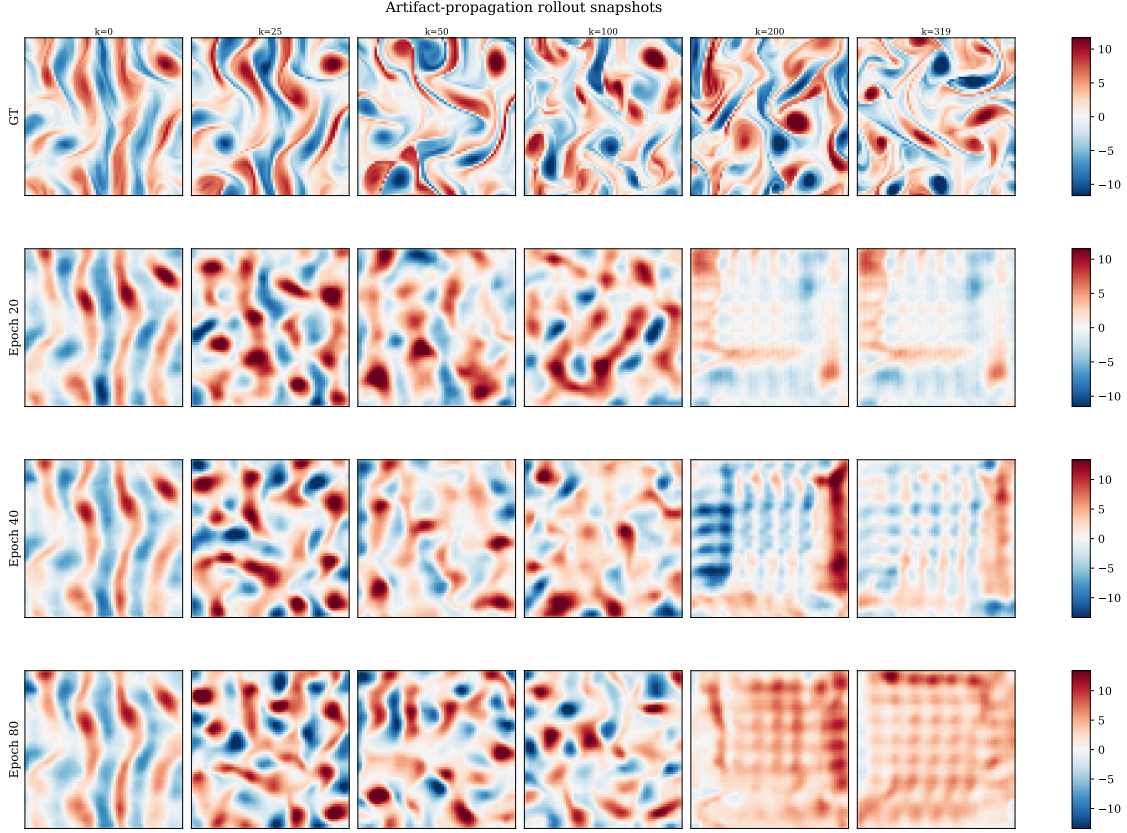


Figure 5.6: Validation rollouts for the artifact-propagation case. The rows compare the ground truth rollout with predictions from epochs 20, 40 and 80 at selected time steps. Unlike the numerical-instability case, the predictions remain bounded, but later frames develop persistent spatial artifacts.

Therefore, the next step was to constrain the learned forward operator more directly.

Orthogonality constraint The next attempt was to replace the spectrum shrinkage workaround with a direct orthogonality constraint on the learned forward operator. Instead of initializing K below the unit circle with `init_scale= 0.9`, the operator was initialized with `init_scale= 1.0` and trained with an additional loss term, $K^\top K - I$, where the aim was not to force an exactly orthogonal matrix, but to encourage the latent dynamics to behave more like a norm-preserving time-evolution step.

This was a useful improvement over the two previous cases. The rollout no longer blew up by orders of magnitude, and it did not collapse into the same persistent grid-like artifacts as shown in Figure 5.7. The predicted fields remained visually flow-like over the full rollout horizon. However, this did not fully solve the AR problem, as the predicted trajectory was still not a satisfying match to the ground-truth trajectory.

Figure 5.7 summarizes the orthogonality-constrained run. Panel (a) shows an epoch-80 rollout sample from the validation set, where compared to the previous cases, the predictions remain bounded and retain better structure. Panel (b) shows the mean rollout MSE over the validation

trajectories across epochs, indicating that the orthogonality loss helps prevent the blow-up behaviour observed earlier. Panel (c) shows that the eigenvalue magnitudes of the learned forward operator stay clustered near the unit circle. Panel (d) shows the training and validation losses, with a more prominent overfitting pattern visible from the worsening validation curve.

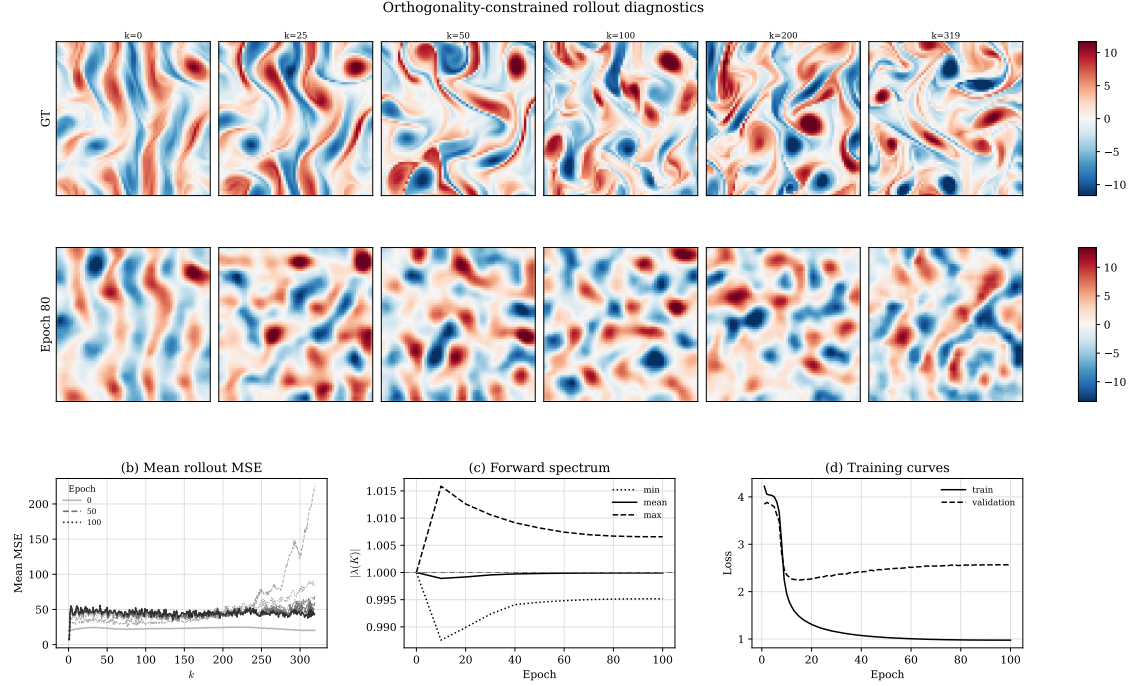


Figure 5.7: Orthogonality constrained AR rollout diagnostics. (a) Ground truth trajectory and epoch-80 prediction at selected time steps. (b) Mean validation rollout MSE across saved epochs. (c) Minimum, mean, and maximum eigenvalue magnitudes of the learned forward operator K . (d) Training and validation loss curves.

This naturally raised a more controlled question, namely if the latent space were already useful, could a finite forward operator K be learned or solved for directly? This question motivates the following least-squares probe.

Before explaining the probe, the training setup was also scaled up. The earlier experiments used a smaller Koopman configuration and a smaller dataset, and the resulting rollouts often looked overly smooth rather than Kolmogorov-like. The new dataset was therefore introduced together with a larger AR model. This change was not treated as a separate controlled experiment, but as a practical setup change before asking whether the latent dynamics could be learned more reliably.

5.2.2 Latent Least Squares Probe

This experiment targeted the linearizability of the learned latent space rather than generalization to unseen trajectories. For each dataset size N , the encoder E and decoder G were trained using reconstruction loss only. The resulting pure weights were then frozen, and latent pairs (z_k, z_{k+1}) were used to solve for K^* by least squares. In this probe, only one forward operator K is considered. As explained previously, a separate backward operator B is not part of the ablation.

The motivation is that using a single K operator makes it easier to diagnose whether the frozen latent space itself can support linear time evolution.

To define the experimental procedure, suppose a fixed dataset size N is given. The encoder E and decoder G are trained until convergence with respect to the reconstruction loss only. Let E_{θ^*} and G_{θ^*} denote the learned pure weights. Using these pure weights, each low-resolution pair is encoded to obtain a latent pair (z_k, z_{k+1}) . Then, using these pure latent pairs, the least-squares solution is fitted for the pure Koopman forward operator K^* . This experiment is repeated for $N = (1, 2, 8, 40)$ using only the new chaotic-regime dataset.

Figure 5.8 shows the rollout MSE for least-squares operators fitted using $N = (1, 2, 8, 40)$ trajectories. For $N = 1$ and $N = 2$, the fitted operator produces low rollout error on the selected trajectories. However, as the number of trajectories increases, the rollout error also rises. For the case of $N = 40$, order-one error is reached rather quickly, suggesting that a single finite-dimensional linear operator can no longer summarize the frozen latent transitions across the larger set of flows.

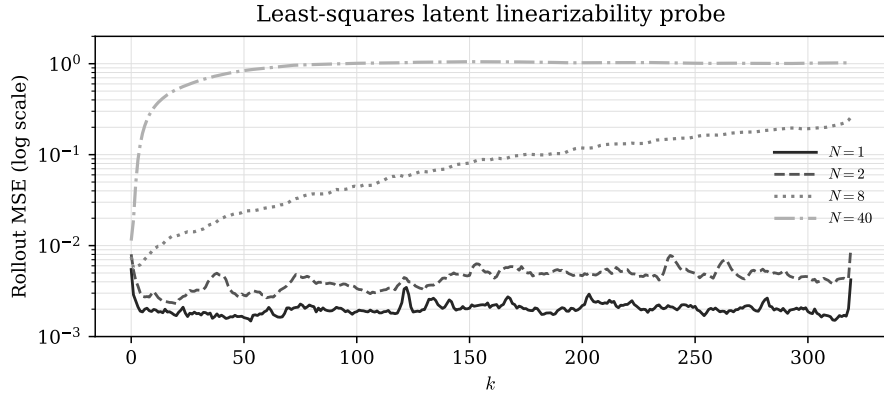


Figure 5.8: Least-squares latent linearizability probe. A single finite-dimensional forward operator K^* is fitted by least squares on frozen reconstruction-only autoencoder latents using increased dataset size with $N = (1, 2, 8, 40)$ training trajectories.

Figure 5.9 shows a qualitative perspective of the same observed problem, where a single operator becomes harder to fit as the dataset size increases. For small $N = 1$, the encoded latent states form a coherent curve in the first two principal components, with a small overlap at a few points where the latents are likely similar. For larger N , increasingly overlapping latent paths are produced. However, this should still not be read as a clean comparison visualization of the latent states because each panel comes from a separately trained autoencoder. It nevertheless suggests why a single operator is much easier to fit in the small-dataset case.

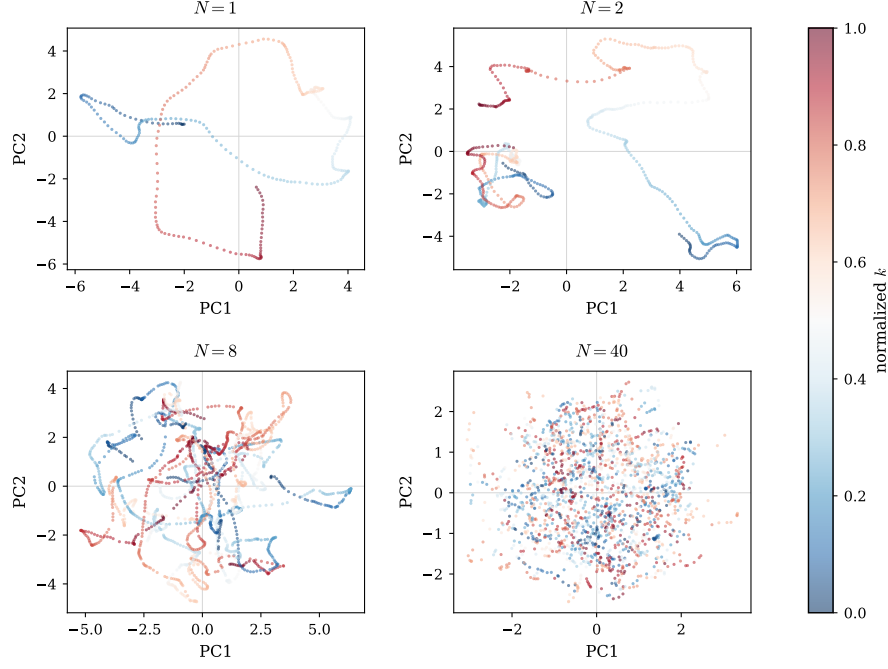


Figure 5.9: Qualitative PCA projections of latent states from independently trained reconstruction-only autoencoders. Points are colored by normalized time step.

Figure 5.10 shows another visualization of the same problem. For small N , most of the latent variance is concentrated in relatively few directions. As N increases, the covariance spectrum becomes flatter, and the cumulative variance rises more slowly. This indicates that the frozen autoencoder uses a broader set of latent directions to represent the larger trajectory set, again making the fit harder as N increases.

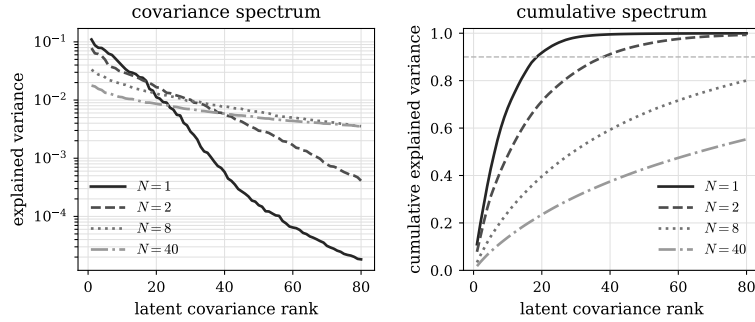


Figure 5.10: Latent covariance spectra for reconstruction-only autoencoders. The left panel shows the normalized covariance eigenvalue spectrum of the encoded latents sorted by rank where the right panel shows the cumulative explained variance.

These observations suggest that the learned latent representation becomes harder to linearize with a single finite-dimensional operator as more trajectories are included. If the least-squares solution already struggles under this frozen representation, then gradient-based learning of K is

also expected to be difficult under the same parameter constraints. Therefore, the next question is whether end-to-end training can improve the single- K formulation.

5.2.3 Ablations under Strict Single-Operator Rollout

The least-squares probe suggested that a frozen autoencoder latent space was not easily summarized by a single finite-dimensional forward operator. The next question was therefore whether end-to-end training could make this strict single-operator formulation viable. In this setting, the encoder, decoder, and Koopman operator are optimized jointly, so the latent representation can adapt to the rollout objective rather than being fixed before K is learned.

All experiments in this subsection use the same strict single-operator regime. The AR model learns one trainable forward operator K , and the backward prediction loss is imposed through $K^\top$ rather than through a separately learned backward operator B . This is combined with reconstruction loss and orthogonality loss on K . The purpose of these experiments is not to claim a fully optimized AR configuration, but to keep the design simple.

The selected checkpoints are taken before clear validation overfitting is observed. This makes the comparison useful for understanding the behaviour of the strict rollout formulation, but a complete conclusion would still require a systematic sweep over model capacity, loss coefficients, and training duration after this subsection.

The first ablation is the strict single-operator baseline. It uses the larger Koopman configuration and the new dataset, while keeping the objective simple with one-step reconstruction, forward prediction, backward prediction, and an orthogonality constraint. The second ablation adds another one-step latent-step consistency loss, which penalizes the difference between $KE(x_k)$ and $E(x_{k+1})$, as explained previously. The third ablation increases the AR capacity within the same strict single-operator regime. The AR width is increased from `ar.dims = 16` to `ar.dims = 32`, giving a model with 16,727,216 parameters. This is the strongest strict attempt in this group, but it is not a clean capacity ablation because training duration is also increased. It should therefore be interpreted as evidence that capacity likely matters in the current set of loss terms, not as proof that capacity alone solves the rollout problem.

Figure 5.11 shows the comparison of strict single-operator AR ablations. Rows correspond to the baseline strict single- K setting, the latent-step consistency variant, and the increased-capacity variant. Columns show training behaviour, Koopman spectral stability, and validation-set rollout statistics.

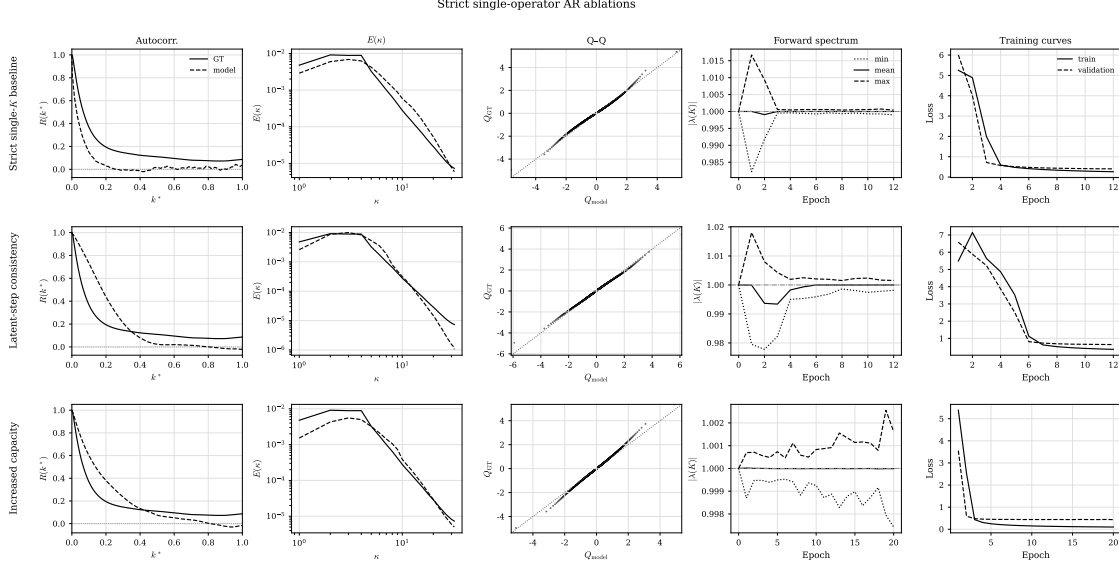


Figure 5.11: Diagnostics comparison of the three ablations. Rows show the strict single- K baseline, the latent-step consistency variant and the increased-capacity variant. Columns compare temporal autocorrelation, energy-spectrum, Q-Q statistics, forward operator spectra, and training curves.

From these ablations, achieving matching autocorrelation, energy, and Q-Q curves remains difficult. The problems may in fact be due to the choice of architecture itself, rather than only the size of the operator or the need for additional loss terms. However, one additional technique remains, that is, reassessing the inference approach itself and asking whether the Koopman inference mode should be used strictly.

Therefore, as an alternative to the Koopman-style method of inference that has been the central focus of this thesis, one additional experiment temporarily reconsiders the rollout procedure. This naturally leads to the next section, where the problem is handled from a more traditional machine-learning perspective, independent of whether the underlying physics constraint is required for producing a coherent rollout over long horizons.

5.2.4 Sequential inference

The strict single-operator ablations show that the difficulty is not only numerical stability, but also maintaining a latent trajectory that remains useful under repeated application of K . The next experiment therefore changes the inference procedure rather than adding another constraint to the strict rollout.

This defines a less strict Koopman-style rollout. Instead of requiring one latent state to remain useful through repeated applications of K , sequential inference reintroduces the encoder at every step, where each predicted frame is decoded back to pixel space and then encoded again before the next application of the latent operator. This makes the rollout less faithful to the strict Koopman interpretation, but it is useful if the goal is to produce a plausible low-resolution trajectory for the later joint pipeline.

Interestingly, the model only required switching to an L_p pixel loss and separate forward/backward

dynamics during training. No latent rollout loss or orthogonality penalty was required.

Figure 5.12 shows a representative sequential rollout and the corresponding test-set diagnostics.

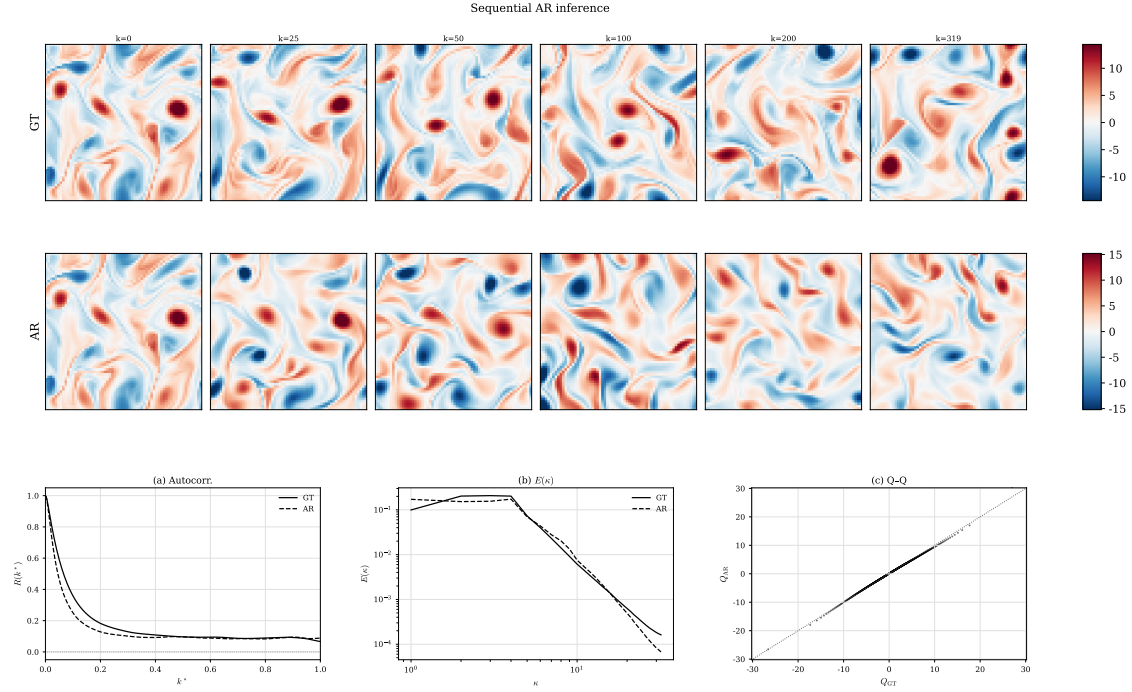


Figure 5.12: Sequential AR inference on the generated test set. The top rows compare ground-truth and AR-predicted low-resolution vorticity snapshots at selected time steps. The bottom row shows temporal autocorrelation, energy-spectrum and Q-Q diagnostics.

This inference method is then carried forward to the joint module, where the produced AR outputs must serve as the low-resolution conditions received by the SR module. This connection is discussed further in Section 5.4.

5.2.5 Additional AR Experiments

The strict single-operator experiments left two practical questions, namely whether weak rollouts were mainly caused by insufficient capacity in the finite-dimensional latent operator and encoder-decoder system, or by the balance between the objective terms used during training. To probe this, a final set of diagnostic AR sweeps was run. These sweeps were not intended as a full hyperparameter optimization, but as screening experiments. The validation autocorrelation score S_R was used as the main scalar summary, while visual rollout quality was kept as a sanity check.

One setting should be made explicit before reading the sweep results. These runs inherited the pixel-space L_p loss used in the sequential-inference experiment. This made the sweeps consistent with the most recent AR training setup, but also limits how cleanly they can be interpreted. The results below therefore compare capacity and loss-weight balance under this specific pixel-loss choice, rather than proving that the same rankings would hold under an MSE-based objective.

The first sweep tests the capacity question directly, as shown in Figure 5.13. In this sweep, the loss weights are fixed, while the encoder-decoder profile and the base width/Koopman dimension

are varied. Each configuration is trained for only 10 epochs, and the three strongest candidates were selected for continuation in the next stage, namely **e1_d048**, with blocks [4, 6, 6, 8], $d = 48$, and $S_R = 0.52$, **e1_d064**, with blocks [4, 6, 6, 8], $d = 64$, and $S_R = 0.74$, and **e2_d064**, with blocks [6, 8, 8, 10], $d = 64$, and $S_R = 0.83$. The strongest Stage 1 result is therefore obtained by the intermediate-depth encoder-decoder at the largest tested Koopman dimension. Increasing the width generally helps, but the deepest encoder-decoder does not improve the result. This suggests that the strict single- K formulation is sensitive to capacity, but not in a simple larger-is-always-better way.

In the Stage 1 heatmap, the notation is kept compact. Each row fixes an encoder-decoder profile, shown by its transformer-block and attention-head configuration. Each column changes the base width of the AR model. Since the latent grid size is fixed, increasing the base width also increases the flattened latent dimension and therefore the size of the learned Koopman matrix K . Each cell reports the validation autocorrelation score S_R on the first line and the total parameter count on the second line.

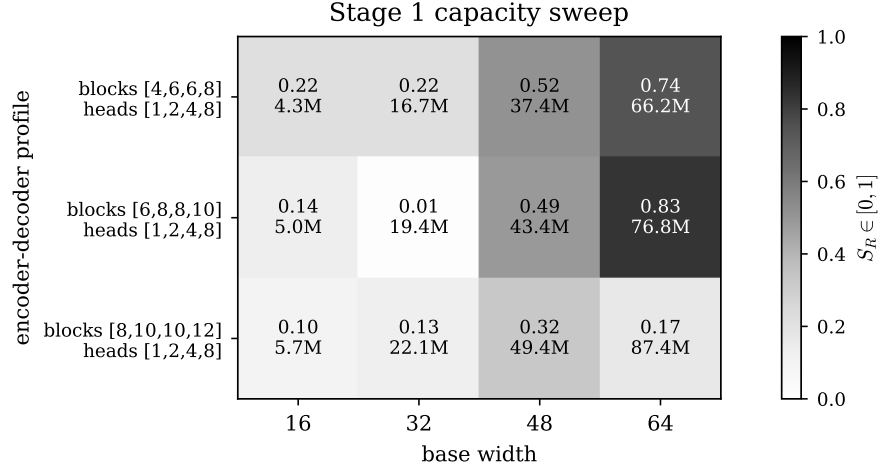


Figure 5.13: Stage 1 capacity sweep for the strict- K AR model. Rows show encoder-decoder profiles, columns show base width, and cells report the validation autocorrelation score S_R above the total parameter count.

The second stage continues with the strongest candidates from the capacity sweep for a longer training horizon, to check whether the rankings differ over increased training time when some profiles may perform better early on. Figure 5.14 shows that the best validation score is not necessarily obtained at the extended final epoch, and may instead worsen over time. Only for the **e1_d048** case was an improved S_R observed early, at epoch 4. For the next stage, the **e2_d064** profile is fixed to screen the role of different loss coefficients.

The Stage 2 heatmap should therefore be read as a continuation check rather than as a new capacity search. The rows are the three selected Stage 1 candidates, the columns are training epochs, and the annotated cell marks the best validation score reached by each candidate.

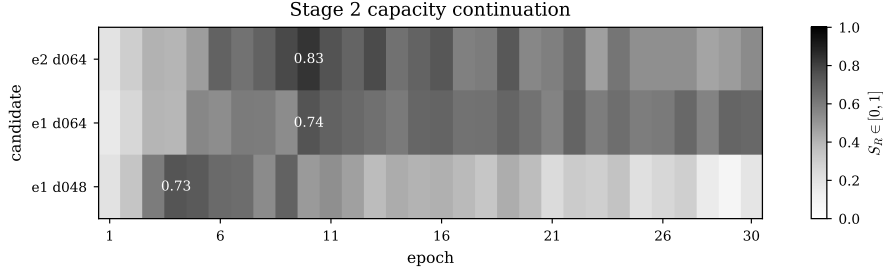


Figure 5.14: Stage 2 continuation sweep for the selected Stage 1 candidates. Rows show candidate capacity settings, columns show epochs, and cells show validation S_R .

The third stage of sweeps keeps the best capacity setting from Stage 2 and varies the coefficients of the forward, backward, reconstruction, latent-forward, and orthogonality losses. The latent-forward one-step loss was excluded previously, but is reintroduced here. For the AR module, the `alpha_e2_d064` profile was selected specifically. For the loss-term coefficients, the forward loss was kept fixed as the main objective with $\lambda_{\text{fwd}} = 1$, while the backward, reconstruction, latent-forward, and orthogonality coefficients were sampled from the discrete set $\{0, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. The sampled sets of loss coefficients are called combinations. Each combination was sampled with a prior check that excluded overlapping and degenerate cases. To ensure that degenerate combinations were not sampled, at least two auxiliary terms had to be active, and at least one stabilizing term among the backward, latent-forward, and orthogonality losses had to be nonzero. The default setting from Stages 1 and 2 was carried over, namely $(\lambda_{\text{fwd}}, \lambda_{\text{bwd}}, \lambda_{\text{recon}}, \lambda_{\text{latent}}, \lambda_{\text{ortho}}) = (1, 1, 1, 1, 10^{-1})$, and was labelled as the default configuration among the other combinations to serve as a reference.

The Stage 3 heatmap keeps the coefficient labels short to avoid overloading the figure. It is mainly used to show when strong validation scores appear during training. The following ranking figure then decodes the corresponding coefficient values for each setting.

Influenced by the Stage 2 results, each configuration was trained for 10 epochs using the pixel-space L_p loss. As shown in Figure 5.15, only a small subset of configurations reached a strong validation autocorrelation score, and the highest score obtained by `gamma_05` occurs mid-training and does not persist to epoch 10. The default configuration, by contrast, reaches a very similar score at the final epoch and visually remains more consistent with the ground-truth autocorrelation trend. Therefore, the main conclusion is not that a better configuration has been found, but that the problem is sensitive to loss weighting and checkpoint choice.

However, the heatmap alone does not show which loss coefficients correspond to each configuration. Therefore, Figure 5.16 complements Stage 3 by showing the same results as a ranked comparison, along with the respective coefficient combination for each achieved score.

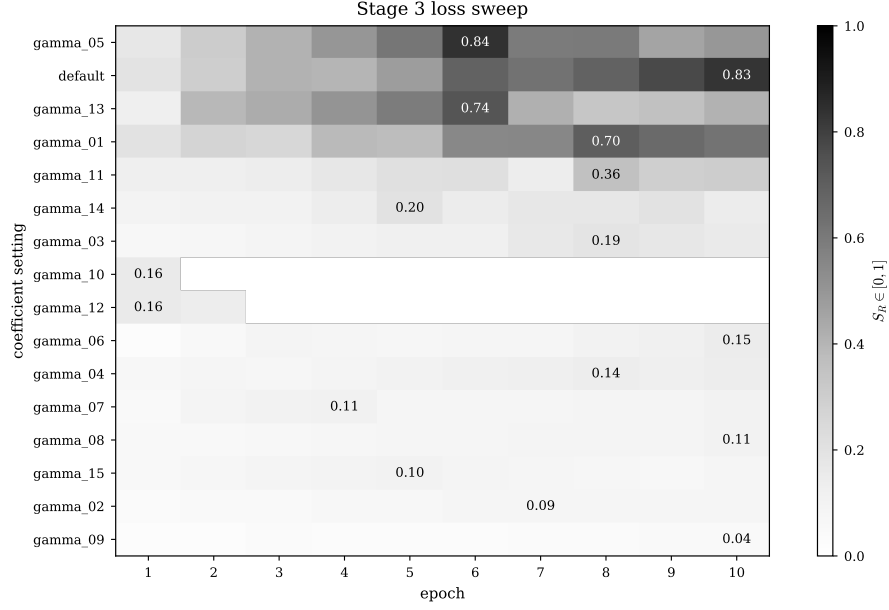


Figure 5.15: Stage 3 loss-coefficient sweep for the selected **alpha_e2_d064** capacity setting. Rows show coefficient configurations sorted by best validation S_R , and columns show epochs.

Additionally, Figure 5.16 shows the ranking of the completed 16 coefficient runs, with the corresponding loss-coefficient combinations shown on the right. Comparing only the strongest two contenders, **gamma_05** and the default configuration, the apparent improvement is very small. The **gamma_05** setting reduces the backward, reconstruction, and orthogonality terms relative to the default, while keeping the forward and latent-forward terms large. This reaches the highest observed peak score, increasing from approximately $S_R = 0.83$ to $S_R = 0.84$, but the improvement is once again negligible. Therefore, this sweep should not be interpreted as finding a clearly better AR model, but rather as showing that the strict single-operator K setup is sensitive to coefficient-weighting choices and may allow earlier convergence while remaining fragile in rollout consistency.

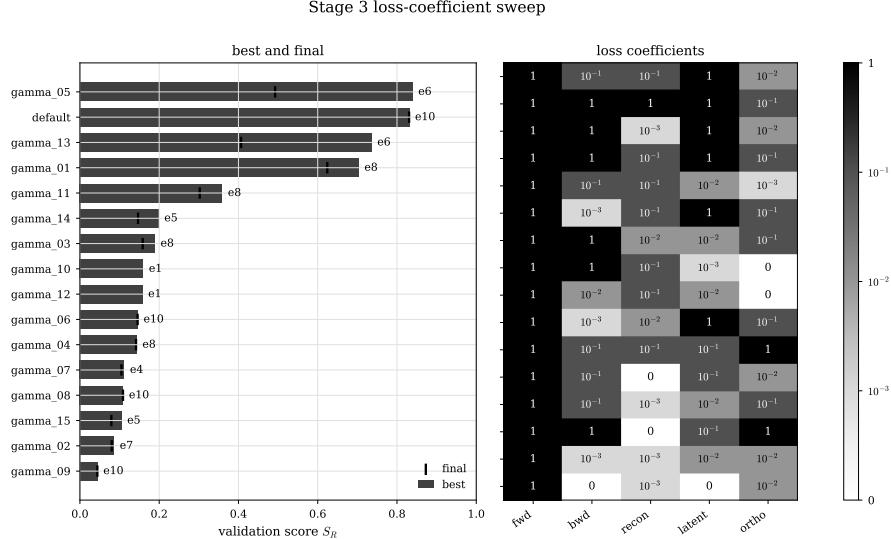


Figure 5.16: Ranking of the Stage 3 loss-coefficient sweep. The left panel compares the best validation S_R with the final-epoch score for each configuration, while the right panel lists the corresponding forward, backward, reconstruction, latent-forward and orthogonality coefficients.

Together, these additional AR results clarify the importance of coefficient weighting as well as the base configuration required to extract meaningful latent representations for the single-operator setting. Capacity is necessary, loss weighting matters, and the best checkpoint can occur before the end of training. However, these experiments also show that the explored set of loss terms and the architecture choice may have contributed to the underwhelming stable-rollout performance.

5.3 Super-Resolution Module Experiments

This section reports the SR module introduced in Section 4.4. Unlike the AR module, the SR module is not evaluated as a temporal rollout model, but as a per-frame conditional reconstruction model. Specifically, given a low-resolution vorticity frame, downsampled by a factor of 4 and then upsampled back to the 256×256 grid, the model generates a high-resolution vorticity sample. Therefore, the purpose of this experiment is to learn a model that can recover fine spatial structure from the coarse upsampled condition before it is used inside the joint pipeline.

The reported model is trained for 100 epochs using the conditional SR3/DDIM configuration described previously. Training uses individual high-resolution frames from the clean KMFlow dataset, with 254, 30, and 30 trajectories assigned to the train, validation, and test sets. This gives 81,280 training frames. The denoiser is a 3.51M-parameter U-Net [Ronneberger et al., 2015] with base width 64, channel multipliers (1, 1, 1, 2), one residual block per resolution, and attention at resolution 16. The model is trained with the DDPM noise-prediction loss, Adam [Kingma and Ba, 2015] with learning rate 10^{-4} , batch size 32, no learning-rate scheduler, and an EMA copy of the weights used for DDIM sampling. Sampling uses 40 DDIM steps from the epoch-100 checkpoint.

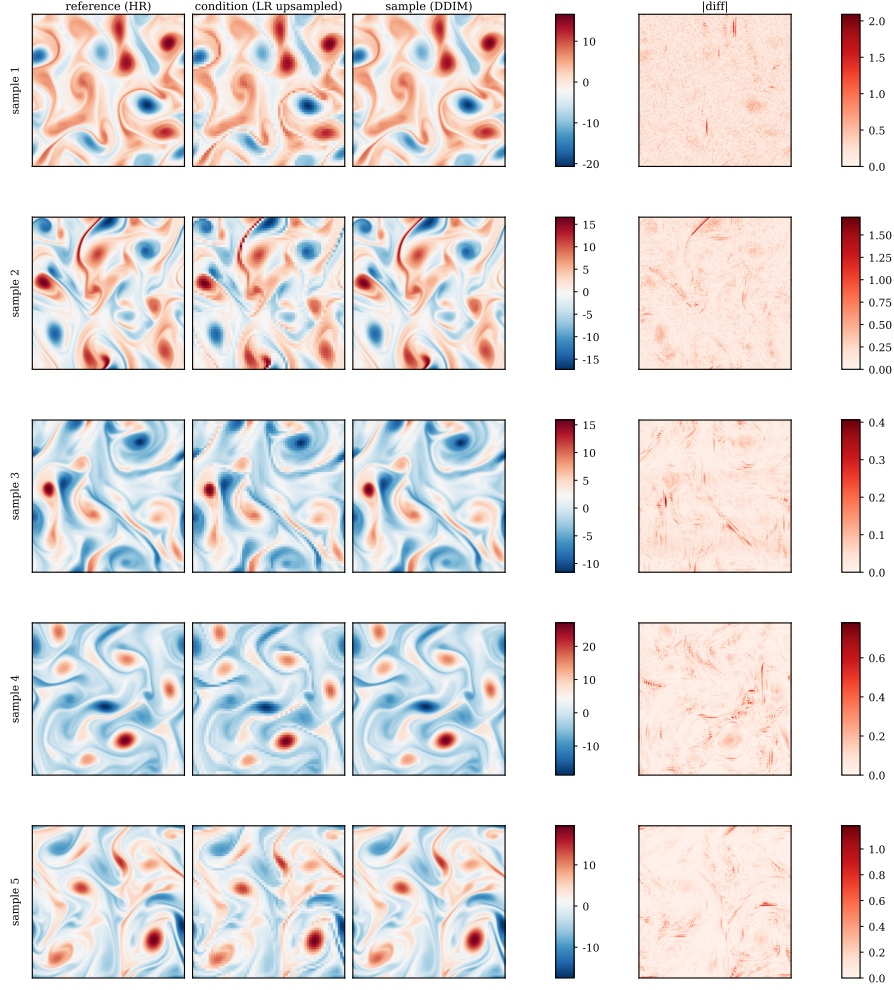


Figure 5.17: SR reconstruction samples. Rows correspond to selected test frames. Columns show the high-resolution reference, the upsampled low-resolution condition, the DDIM super-resolution sample, and the absolute error.

Figure 5.17 shows five example samples randomly selected from the test set. Each row compares the high-resolution reference at 256×256 , the conditional frame obtained by first downsampling the high-resolution frame to 64×64 and then upsampling it back to 256×256 , and the generated DDIM sample at 256×256 . The rightmost column shows the absolute error between the reference and the DDIM sample. The generated DDIM samples recover the large coherent vorticity patterns visible in the reference while improving over the coarse upsampled condition. However, the remaining error is mainly located around sharp filaments and small-scale structures, giving a tolerable visual assessment.

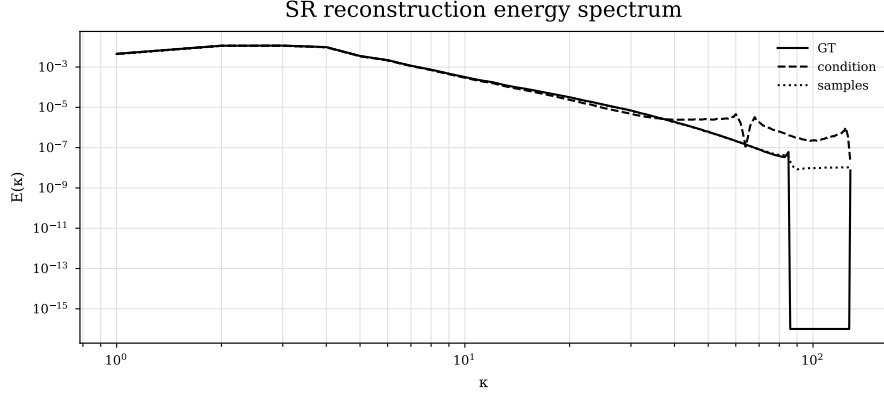


Figure 5.18: Energy-spectrum diagnostic for the SR module on selected test-set frames. The curves compare the high-resolution reference, the upsampled low-resolution condition, and the DDIM super-resolution sample.

In Figure 5.18, the energy-spectrum comparison is made between the reference frames, the upsampled low-resolution conditions, and the DDIM samples. Specifically, the DDIM samples are averaged over 64 random test frames instead of the full test set, and the sampled frames follow the reference spectrum well over the dominant low- and mid-wavenumber range. The largest mismatch appears at the tail, which is consistent with the absolute error maps in Figure 5.17.

Taken together, the standalone SR experiment shows that diffusion can act as a conditional spatial reconstruction model under clean low-resolution guidance. The DDIM samples recover the main coherent vorticity structures from the upsampled condition, while the remaining errors are mostly visible around high-wavenumber patterns, as also evident in the energy-spectrum evaluation. This means that the SR module is able to refine coarse vorticity frames, but only relative to the condition given. Therefore, the remaining question is whether this behaviour still holds when the low-resolution condition is no longer obtained by direct downsampling of the ground truth, but is instead provided by the predicted rollout. This motivates the final joint AR-SR experiment.

5.4 Joint Module Experiments

This section reports the final joint AR-SR experiment, as previously described in Section 4.5. No additional form of training is introduced here. Instead, the trained sequential-inference checkpoint from Section 5.2.4 and the trained SR checkpoint from Section 5.3 are connected at inference time. The process begins by using a starter low-resolution frame for the AR module to generate a rollout of a chosen length. Each predicted low-resolution frame is then upsampled and used as a condition for the SR module. The SR module therefore carries the risk of an out-of-distribution problem when it receives frames from the AR module.

The joint experiment is organized into two diagnostic parts. First, a selected test trajectory is shown through a low-resolution reference rollout, a low-resolution AR rollout, and the corresponding high-resolution AR-SR rollout, along with evaluation plots. Second, the inference is extended to a longer rollout of 1024 frames, beyond the original 320-frame data horizon, to check whether visual coherence is maintained beyond the original rollout window. Additionally, the SR

module uses the EMA diffusion model and 40 DDIM sampling steps, matching the standalone SR experiment.

Figure 5.19 shows the main diagnostic. Although matching the ground-truth trajectory is not expected over long horizons, coherent rollouts are produced in low-resolution space by the AR module, and the up-resolution counterparts do not break the original flow structure in the displayed frames. The autocorrelation curves therefore follow the ground-truth average closely, while the Q–Q comparison is reasonable and the energy spectrum remains close over the main range but differs near the tail.

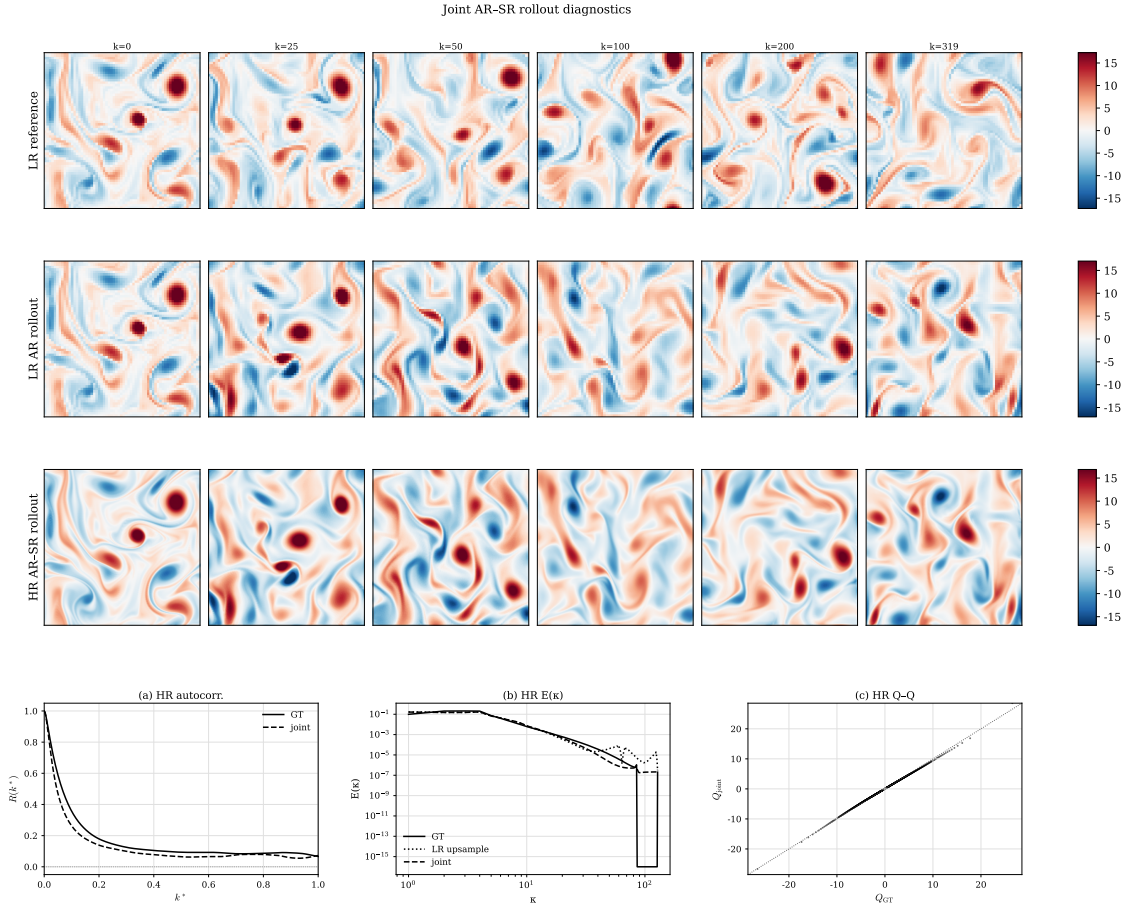


Figure 5.19: Diagnostics for the joint AR–SR pipeline. The image rows show the low-resolution reference rollout, the low-resolution AR rollout, and the high-resolution AR–SR rollout. The lower panels report high-resolution autocorrelation, energy-spectrum $E(\kappa)$, and Q–Q diagnostics over the full rollout.

Figure 5.20 shows the qualitative long-horizon extension on a single held-out trajectory. Since it is extended to 1024 frames, it should not be read as a pointwise accuracy test against the ground truth, which has a shorter rollout length, but rather as a check of whether the modular pipeline is able to stay stable over long horizons.

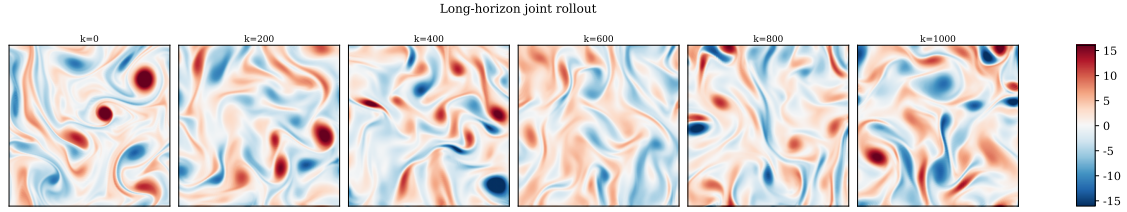


Figure 5.20: Long-horizon rollout from the joint AR–SR pipeline. The panels show selected high-resolution vorticity fields generated by applying the SR module to the AR rollout at increasing time steps.

Taken together, the joint experiment shows that the modular AR–SR pipeline can be executed end-to-end and can produce high-resolution vorticity frames from predicted low-resolution rollouts. The main limitation remains the temporal aspect, where the AR module needs to produce rollouts that remain within the input distribution expected by the SR module. As shown in this thesis, the most practical AR rollout is not fully strict in its use of the Koopman approach.

Chapter 6

Discussion and Limitations

The aim of this thesis was to study a modular surrogate model for two-dimensional Kolmogorov flow, where temporal prediction is handled by an autoregressive (AR) module and spatial reconstruction is handled by a super-resolution (SR) module. The central restriction was placed on the AR module, which was required to use a Koopman-style finite latent operator. The experiments showed that the full AR-SR pipeline can be executed end-to-end, but they also showed that the main difficulty remains the temporal part of the problem.

The AR experiments suggested that stability is not the same as correctness, which was evident from early strict Koopman rollouts either becoming numerically unstable or developing persistent spatial artifacts. Adding an orthogonality constraint helped keep the learned operator bounded and made the rollouts more visually coherent, but it did not fully solve the long-horizon prediction problem. The later capacity and loss-weight sweeps also showed that model size, coefficient choices, and checkpoint selection all affected the result. Even then, the strict single-operator formulation remained fragile, suggesting that the difficulty is not only a matter of optimization, but also of structure, namely, one finite latent matrix is a strong constraint for chaotic flow dynamics.

The sequential-inference experiment gave a more practical rollout, but at the cost of weakening the strict Koopman interpretation. By decoding and re-encoding at every step, the model no longer depended only on repeated application of one latent operator. This helped keep the predicted trajectory closer to the space seen by the encoder and decoder, but it also meant that the nonlinear encoder-decoder pair became part of the time-stepping process, thus introducing additional nonlinearity and moving away from the strict Koopman formulation.

One useful way to interpret these results is as a compatibility problem between learned spaces. Specifically, the AR module must not only predict a reasonable next low-resolution frame, but also keep its rollout near the set of states that the encoder, decoder, and SR module have learned to process. In latent space, repeated application of the Koopman operator must also keep the state close to the encoder’s learned latent representation space. When this condition fails, the decoded or super-resolved field may still look plausible, but it is not necessarily following the same physical evolution. This explains why bounded spectra, low reconstruction loss, and visually coherent frames are necessary but not sufficient for reliable long-horizon rollout.

In contrast, the SR module behaved more cleanly than the AR module. When the low-resolution condition was obtained from the data, the conditional diffusion model was able to

reconstruct the main large-scale and mid-scale vorticity structures at high resolution, while the remaining errors were mostly located around sharper filaments and high-wavenumber features. This is consistent with the role assigned to the SR module, that is, it is not responsible for learning temporal dynamics, but for recovering spatial detail from a coarse condition.

The joint experiment combines these two behaviours. The AR-SR pipeline can produce high-resolution vorticity frames from predicted low-resolution rollouts, and the long-horizon samples remain visually coherent. However, the quality of the joint pipeline is still limited by the AR states passed into the SR module. If the AR rollout drifts away from the distribution of clean low-resolution conditions, the SR module can refine the frame but cannot fully repair the temporal error. Thus, the modular decomposition is useful, but it does not remove the need for a strong temporal model.

The main limitations of this thesis follow from this structure, where most AR training objectives were based on one-step losses, while the models were evaluated through multistep rollout. The validation autocorrelation score was used as the main scalar summary, even though the energy spectrum and Q-Q diagnostics were also important for interpretation. This thesis also lacked a full baseline comparison against models such as FNO, direct regression methods for the AR module, and deterministic upsampling methods for the SR module.

These limitations suggest a few natural future directions, where the most direct one is to relax the strict single-operator assumption. Instead of forcing the whole learned latent space to evolve under one global matrix, future work could study local, switching, or mixture-based Koopman models, where different operators describe different regions of the latent dynamics. A second direction is to train the temporal model closer to how it is used at inference, for example with multistep latent or pixel losses and with spectral, energy, or enstrophy-based terms that better reflect turbulent flow statistics. Because some terms could conflict at certain stages of the AR training process, a form of curriculum method could be employed, where a separate model adjusts the coefficients during training. Finally, the AR module could be assigned to learn at the data-native resolution, completely removing the need for a diffusion super-resolution module, or the decoder part of the AR module could be removed and connected to a latent diffusion scheme to train an AR-SR Koopman autoencoder with latent diffusion to the desired resolution, allowing the system to be trained end-to-end.

Chapter 7

Conclusion

This thesis investigated a modular AR–SR surrogate model for two-dimensional Kolmogorov-flow prediction of future vorticity frames. The central question was whether the low-resolution temporal module could be represented by a Koopman-style autoencoder under the strict constraints of a single operator and one-step training losses. In addition, an up-resolution module was investigated to test whether predicted rollouts could be upsampled through a conditional diffusion model.

The results showed that the diffusion SR module can recover coherent high-resolution vorticity structures from clean low-resolution conditions, while the AR module remains the bottleneck. Strict single-operator Koopman rollouts were difficult to make reliable over long horizons, although orthogonality constraints improved stability, and capacity and loss-weight choices affected performance. However, these changes did not fully remove the fragility of the temporal model, while the sequential-inference version produced the most useful rollouts.

Overall, this thesis shows that the AR–SR decomposition is a promising way to separate temporal prediction from spatial reconstruction, but that the temporal problem remains the central scientific obstacle. A finite Koopman-style operator can provide structure, stability, and interpretability, yet chaotic Kolmogorov-flow rollouts require more than bounded linear propagation in latent space. The main contribution of the thesis is therefore both constructive and diagnostic, since it builds an end-to-end AR–SR pipeline and identifies where the strict Koopman assumption becomes fragile when placed inside that pipeline.

Appendix A

Use of AI Tools

I used AI assistants throughout the preparation of this thesis. In particular, they helped me navigate the space of possible experiments, work through the mathematical and physical background, and shape parts of the experimental procedure. I treated their output as a starting point rather than a conclusion and have made the effort to verify, test and rework code implementations that were generated by AI in the process. I recognize the boundary between “assisted” and “authored” is very thin and have remained thoughtful about that distinction as all narrative work, experimental results, and creative work were handled by me and chosen with meticulous precision such that it would not hurt the thesis in any way. All external sources were cited following standard academic practice. I take full responsibility for this thesis, including any errors, omissions, or misinterpretations that remain, regardless of whether AI tools were included in the process that produced them; during proofreading (explicitly to keep my narrative and word choice), grammatical inconsistencies (via deterministic tools) and earlier discovered mathematical notation inconsistencies that would have caused confusion to the reader were detected and refined.

Bibliography

- Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael W. Mahoney. Forecasting sequential data using consistent Koopman autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 475–485. PMLR, 2020. URL <https://proceedings.mlr.press/v119/azencot20a.html>.
- Paolo Boscariol, Domenico Dona’, Dario Richiedei, and Alberto Trevisani. Deep-Koopman-enhanced Kalman filter for multibody systems. *Multibody System Dynamics*, 2025. doi: 10.1007/s11044-025-10108-7.
- Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control. *PLOS ONE*, 11(2):e0150171, 2016. doi: 10.1371/journal.pone.0150171.
- Marko Budišić, Ryan M. Mohr, and Igor Mezić. Applied Koopmanism. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 22(4):047510, 2012. doi: 10.1063/1.4772195.
- Stephen A. Falconer, David J. B. Lloyd, and Naratip Santitissadeekorn. Tracking and forecasting oscillatory data streams using Koopman autoencoders and Kalman filtering. *Physica D: Nonlinear Phenomena*, 476:134649, 2025. doi: 10.1016/j.physd.2025.134649.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL <https://www.deeplearningbook.org/>.
- Rares Grozavescu, Pengyu Zhang, Mark Girolami, and Etienne Meunier. Scaling continuous-time Koopman autoencoders for high-dimensional fluid dynamics. *arXiv preprint arXiv:2602.02832*, 2026. URL <https://arxiv.org/abs/2602.02832>.
- Rakesh Halder, Benet Eiximeno, and Oriol Lehmkuhl. Reduced-order modeling of large-scale turbulence using Koopman β -variational autoencoders. *arXiv preprint arXiv:2507.18754*, 2025. URL <https://arxiv.org/abs/2507.18754>.
- Amir Pouya Hemmasian and Amir Barati Farimani. Coupled diffusion-encoder models for reconstruction of flow fields. *arXiv preprint arXiv:2601.07946*, 2026. URL <https://arxiv.org/abs/2601.07946>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.

- Eurika Kaiser, J. Nathan Kutz, and Steven L. Brunton. Data-driven discovery of Koopman eigenfunctions for control. *Machine Learning: Science and Technology*, 2(3):035023, 2021. doi: 10.1088/2632-2153/abf0f5.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- Stefan Klus, Feliks Nüske, Péter Koltai, Hao Wu, Ioannis Kevrekidis, Christof Schütte, and Frank Noé. Data-driven model reduction and transfer operator approximation. *Journal of Nonlinear Science*, 28:985–1010, 2018. doi: 10.1007/s00332-017-9437-7.
- Georg Kohl, Li-Wei Chen, and Nils Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. *Neural Networks*, 199:108641, 2026. doi: 10.1016/j.neunet.2026.108641.
- B. O. Koopman. Hamiltonian systems and transformations in hilbert space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, 1931. doi: 10.1073/pnas.17.5.315.
- B. O. Koopman and J. von Neumann. Dynamical systems of continuous spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, 1932. doi: 10.1073/pnas.18.3.255.
- Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018. doi: 10.1016/j.automatica.2018.03.046.
- Qianxiao Li, Felix Dietrich, Erik M. Bollt, and Ioannis G. Kevrekidis. Extended dynamic mode decomposition with dictionary learning: A data-driven adaptive spectral decomposition of the Koopman operator. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 27(10), 2017. doi: 10.1063/1.4993854.
- Yunzhu Li, Hao He, Jiajun Wu, Dina Katabi, and Antonio Torralba. Learning compositional Koopman operators for model-based control. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=H1ldzA4tPr>.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=c8P9NQVtmn0>.
- Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM / IMS J. Data Sci.*, 1, 2024. doi: 10.1145/3648506. URL <https://doi.org/10.1145/3648506>.
- Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9:4950, 2018. doi: 10.1038/s41467-018-07210-0.

- Andreas Mardt, Luca Pasquali, Hao Wu, and Frank Noé. VAMPnets for deep learning of molecular kinetics. *Nature Communications*, 9, 2018. doi: 10.1038/s41467-017-02388-1.
- Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiaming Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations*, 2022.
- Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.
- Igor Mezić and Andrzej Banaszuk. Comparison of systems with complex behavior. *Physica D: Nonlinear Phenomena*, 197(1–2):101–133, 2004. doi: 10.1016/j.physd.2004.06.015.
- L. Molgedey and H. G. Schuster. Separation of a mixture of independent signals using time delayed correlations. *Physical Review Letters*, 72:3634–3637, 1994. doi: 10.1103/PhysRevLett.72.3634.
- Jeremy Morton, Freddie D. Witherden, Antony Jameson, and Mykel J. Kochenderfer. Deep dynamical modeling and control of unsteady fluid flows. In *Advances in Neural Information Processing Systems*, volume 31, pages 9258–9268, 2018.
- Indranil Nayak, Ananda Chakrabarti, Mrinal Kumar, Fernando L. Teixeira, and Debdipta Goswami. Temporally consistent Koopman autoencoders for forecasting dynamical systems. *Scientific Reports*, 15:22127, 2025. doi: 10.1038/s41598-025-05222-7.
- Frank Noé and Feliks Nüske. A variational approach to modeling slow processes in stochastic dynamical systems. *Multiscale Modeling & Simulation*, 11(2):635–655, 2013. doi: 10.1137/110858616.
- Feliks Nüske, Bettina G. Keller, Guillermo Pérez-Hernández, Antonia S. J. S. Mey, and Frank Noé. Variational approach to molecular kinetics. *Journal of Chemical Theory and Computation*, 10(4):1739–1752, 2014. doi: <https://doi.org/10.1021/ct4009156>.
- Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.
- B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992. doi: 10.1137/0330046.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.
- Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009. doi: 10.1017/S0022112009992059.

- Chitwan Saharia, Jonathan Ho, William Chan, Tim Salimans, David J. Fleet, and Mohammad Norouzi. Image super-resolution via iterative refinement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4):4713–4726, 2023. doi: 10.1109/TPAMI.2022.3204461.
- Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. doi: 10.1017/S0022112010001217.
- Dule Shu, Zijie Li, and Amir Barati Farimani. A physics-informed diffusion model for high-fidelity flow field reconstruction. *Journal of Computational Physics*, 478:111972, 2023. doi: 10.1016/j.jcp.2023.111972.
- Jeffrey P. Slotnick, Abdollah Khodadoust, Juan Alonso, David Darmofal, William Gropp, Elizabeth Lurie, and Dimitri J. Mavriplis. CFD vision 2030 study: A path to revolutionary computational aerosciences. Technical Report NASA/CR-2014-218178, NASA, 2014. URL <https://ntrs.nasa.gov/citations/20140003093>.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=St1giarCHLP>.
- Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning Koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems*, pages 1130–1140, 2017.
- Christoph Wehmeyer and Frank Noé. Time-lagged autoencoders: Deep learning of slow collective variables for molecular kinetics. *The Journal of Chemical Physics*, 148(24), 2018. doi: 10.1063/1.5011399.
- Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015. doi: 10.1007/s00332-015-9258-5.
- Xingjian Wu, Xiangfei Qiu, Hongfan Gao, Jilin Hu, Bin Yang, and Chenjuan Guo. K^2 VAE: A Koopman-Kalman enhanced variational autoencoder for probabilistic time series forecasting. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 67562–67583. PMLR, 2025. URL <https://icml.cc/virtual/2025/poster/46346>.
- Xiao Xue, Tianyue Yang, Mingyang Gao, Leyu Pan, Maida Wang, Kewei Zhu, Shuo Wang, Jiuling Li, Marco F. P. ten Eikelder, and Peter V. Coveney. Uni-Flow: A unified autoregressive-diffusion model for complex multiscale flows. *arXiv preprint arXiv:2602.15592*, 2026. URL <https://arxiv.org/abs/2602.15592>.
- Enoch Yeung, Soumya Kundu, and Nathan O. Hodas. Learning deep neural network representations for Koopman operators of nonlinear dynamical systems. In *2019 American Control Conference (ACC)*, 2017. URL <https://arxiv.org/abs/1708.06850>.

Syed Waqas Zamir, Aditya Arora, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Restormer: Efficient transformer for high-resolution image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5728–5739, 2022.